Document number: P0355R2

Howard E. Hinnant
2017-02-04

Extending <chrono> to Calendars and Time Zones

Contents

Revision History

Changes since R1

- Make `time_point` incremental and decrementable
- Add unary operators + and - to `year`
- Remove `enum {am, pm}`, `time_of_day` constructors which use it, and `make_time` factory functions which use it.
- Add `to_stream`.
- Add `format` and `parse` for `duration`.

Changes since R0

- Tighten up `utc_clock::utc_to_sys` and `utc_clock::sys_to_utc`.
- Eliminate `utc_clock::utc_to_sys` and `utc_clock::sys_to_utc` in favor of free functions such as `to_sys_time` and `to_utc_time`.
- Give `utc_time` a streaming operator.
- Change `format` to take `time_point`s by `const&` instead of by value.
- Add trivial default constructors to most calendar types.
- Create %Ez & %Oz to put ':' in offset for format and parse.
- Add %F to parse.
- Changed parse functions to parse manipulators.
- Excuse local_t from being a clock
- Guarantee Unix Time.
- Add duration stream insertion.
- Introduce tai_clock.
- Introduce gps_clock.
- Removed `noexcept` from `make_time`.

Introduction

The purpose of a calendar is to give a name to each day.[1] There are many different ways this can be accomplished. This paper proposes only the Gregorian calendar. However the design of this proposal is such that clients can code other calendars and have them interoperate with `<chrono>`, the civil calendar, and with time zones, all with a minimal coupling. For example:

```
#include "coptic.h"  // not proposed, just an example
#include <chrono>
#include <iostream>

int
main()
{
    using namespace std::chrono_literals;
    auto date = 2016y/may/29;
    cout << date << " is " << coptic::year_month_day{date} << " in the Coptic calendar\n";
    // 2016-05-29 is 1732-09-21 in the Coptic calendar
}
```

The above example creates a date in the Gregorian calendar (proposed) with the literal `2016y/may/29`. The meaning of this literal is without question. It is conventional and clearly readable. This proposal has no knowledge whatsoever of the Coptic calendar. However it is relatively easy to create a Coptic calendar (which knows nothing about the Gregorian calendar), which will convert to and from the Gregorian calendar. This is done by establishing a clear and simple communication channel between calendar systems and the `<chrono>` library (specifically a `system_clock::time_point` with a precision of days).

The paper proposes:

1. Minimal extensions to `<chrono>` to support calendar and time zone libraries.
2. A proleptic Gregorian calendar, hereafter referred to as the *civil* calendar.
3. A time zone library based on the IANA Time Zone Database.
4. `strftime`-like formatting and parsing facilities with fully operational support for fractional seconds, time zone abbreviations, and UTC offsets.
5. A `<chrono>` clock for computing with leap seconds which is also supported by the IANA Time Zone Database.

Everything proposed herein has been fully implemented here:

> https://github.com/HowardHinnant/date

The implementation includes full documentation, and an active community of users with positive field experience. The implementation has been ported to Windows, Linux, and OS X.

The API stresses:

- Seamless integration with the existing `<chrono>` library.
- Type safety.
- Detection of errors at compile time.
- Performance.
- Ease of use.
- Readable code.
- No artificial restrictions on precision.

Listing "Performance" in the API design deserves a little explanation as one usually thinks of that as an implementation issue. Think of it this way:

- If `vector<T>::push_front(const T&)` existed, that would encourage inefficient code, even though it would be trivial to implement.
- If `list<T>::operator[](size_type index)` existed, that would encourage inefficient code, even though it would be trivial to implement (by incrementing from `begin()` or decrementing from `end()`).

This API makes it convenient to write efficient code, and inconvenient to write inefficient code. It turns out that conversion between a field type such as `{year, month, day}` and a serial type such as `{count-of-days}` is one of the more expensive operations when dealing with calendrical computations. Both data structures are very useful (just as both `vector` and `list` are very useful). So this library puts *you* in control of when and how often that conversion takes place, and makes it easy to avoid such conversions when not necessary.

Description

One can create a `year` like this:

```
auto y = year{2016};
```

Just like `<chrono>`, type safety is taken very seriously. The type `year` is distinct from type `int`, just as `3` can never mean "3 seconds", unless it is *explicitly* typed to do so: `seconds{3}`.

And just like `seconds`, there is a year literal suffix which can help make your code more readable:

```
auto y = 2016y;
```

`year` is a *partial-calendar-type*. It can be combined with other partial-calendar-types to create a *full-calendar-type* such as `year_month_day`. Full-calendar-types can be converted to and from the family of `system_clock::time_point`s. Full-calendar-types such as `year_month_day` are time points with a precision of a day, but they are also *field* types. They are composed of 3 fields under the hood: `year`, `month` and `day`. Thus when you construct a `year_month_day` from a `year`, `month` and `day`, absolutely no computation takes place. The only thing that happens is a `year`, `month` and `day` are stored inside the `year_month_day`.

```
year_month_day ymd1{2016y, month{5}, day{29}};
```

This is a *very* simple operation and can even be made `constexpr` when all of the inputs are compile-time constants. And *conventional syntax* is available which means the exact same thing, with the same run-time or compile-time performance. It can make date literals much more readable without sacrificing type safety:

```
constexpr year_month_day ymd1{2016y, month{5}, day{29}};
constexpr auto ymd2 = 2016y/may/29d;
static_assert(ymd1         == ymd2);
static_assert(ymd1.year()  == 2016y);
static_assert(ymd1.month() == may);
static_assert(ymd1.day()   == 29d);
```

`year_month_day` is a *very* simple, *very* understandable *calendrical* data structure:

```
class year_month_day
{
    chrono::year  y_;  // exposition only
    chrono::month m_;  // exposition only
    chrono::day   d_;  // exposition only

public:
    constexpr year_month_day(const chrono::year& y, const chrono::month& m, const chrono::day& d) noexcept;
    // ...
```

By now you should be yawning and muttering "so what?"

Now we introduce a little `<chrono>` infrastructure that serves as the communication channel with simplistic calendrical data structures such as `year_month_day`.

```
using days = duration<int32_t, ratio_multiply<ratio<24>, hours::period>>;
template <class Duration> using sys_time = time_point<system_clock, Duration>;
using sys_days = sys_time<days>;
```

`sys_days` **is** a `std::chrono::time_point`. This `time_point` is based on `system_clock` and has a very coarse precision: 24 hours. Just as `system_clock::time_point` is nothing more than a count of microseconds (or nanoseconds, or whatever), `sys_days` is simply a count of days since the `system_clock` epoch. And `sys_days` is *fully* interoperable with `system_clock::time_point` in all of the ways normal to the `<chrono>` library:

- `sys_days` implicitly converts to `system_clock::time_point` with no truncation error.
- `system_clock::time_point` *does not* implicitly convert to `sys_days` because it would involve truncation error.
- Explicit conversion can be achieved from `system_clock::time_point` by using the existing `<chrono>` facilities `time_point_cast` or `floor`.

```
constexpr system_clock::time_point tp = sys_days{2016y/may/29d}; // Convert date to time_point
static_assert(tp.time_since_epoch() == 1'464'480'000'000'000us);
constexpr auto ymd = year_month_day{floor<days>(tp)};            // Convert time_point to date
static_assert(ymd == 2016y/may/29d);
```

The calendrical type `year_month_day` provides conversions to and from `sys_days`. This conversion is easy to do for std::lib implementors using algorithms such as these. If the committee standardizes existing practice and specifies that `system_clock` measures Unix Time, then it will be equally easy for anyone to write their own calendar system which converts to and from `sys_days` (e.g. the coptic example in the introduction).

This proposal actually contains a second calendar. It is so closely related to the civil calendar that we normally don't think of it as another calendar. We often refer to dates like "the 5th Sunday of May in 2016" as opposed to "the 29th of May in 2016." This proposal makes it so easy to build fully functional calendars that interoperate with `system_clock::time_point`, that it is nearly trivial to include such functionality:

```
constexpr system_clock::time_point tp = sys_days{sun[5]/may/2016}; // Convert date to time_point
static_assert(tp.time_since_epoch() == 1'464'480'000'000'000us);
constexpr auto ymd = year_month_weekday{floor<days>(tp)};          // Convert time_point to date
static_assert(ymd == sun[5]/may/2016);
```

The literal `sun[5]/may/2016` means "the 5th Sunday of May in 2016." The *conventional* syntax is remarkably readable. Constructor syntax is also available to do the same thing. The type constructed is `year_month_weekday` which does nothing but store a `year`, `month`, `weekday`, and the number 5. This "auxiliary calendar" converts to and from `sys_days` just like `year_month_day` as demonstrated above. As such, `year_month_weekday` will interoperate with `year_month_day` (by bouncing off of `sys_days`) just as it will with any other calendar that interoperates with `sys_days`:

```
static_assert(2016y/may/29d == year_month_day{sun[5]/may/2016});
```

Since `year_month_day` is so easy to convert to (or from) a `time_point` it makes sense to convert to a `time_point` when you need to talk about a date and time-of-day:

```
constexpr auto tp = sys_days{2016y/may/29d} + 7h + 30min; // 2016-05-29 07:30 UTC
```

The time zone is implicitly UTC because `system_clock` tracks Unix Time which is (a very close approximation to) UTC. If you need another time zone, no worries, we'll get there. And remember, `tp` above is a `system_clock::time_point`, except with minutes precision. You can compare it with `system_clock::now()` to find out if the date is in the past or the future. Also note that the syntax above (like `<chrono>`) is *precision neutral*. That's because the syntax above **is** `<chrono>`, except for the part converting a calendar type into the `<chrono>` system. If you suddenly need to convert your minutes-precision time point into seconds or milliseconds (or whatever) precision, the change is seamlessly handled by the existing `<chrono>` system:

```
constexpr auto tp = sys_days{2016y/may/29d} + 7h + 30min + 6s + 153ms; // 2016-05-29 07:30:06.153 UTC
```

Simple streaming is provided:

```
cout << tp << '\n';  // 2016-05-29 07:30:06.153
```

But I need the time in Tokyo!

```
auto tp = sys_days{2016y/may/29d} + 7h + 30min + 6s + 153ms; // 2016-05-29 07:30:06.153 UTC
auto zt = make_zoned("Asia/Tokyo", tp);
cout << zt << '\n';                                          // 2016-05-29 16:30:06.153 JST
```

The helper function `make_zoned` creates a type `zoned_time` which is templated on the duration type of `tp`. The use of `make_zoned` *deduces* that you desire milliseconds precision in this example. This effectively pairs a time zone with a time point. In this example we pair the time zone "Asia/Tokyo" with a `sys_time` (which is implicitly UTC). When printed out, you see the local time, and by default the current time zone abbreviation. Also by default, you see the full precision of the `zoned_time`.

Sometimes, instead of specifying the time in UTC as above, it is convenient to specify the time in terms of the local time of the time zone. It is very easy to change the above example to mean 7:30 JST instead of 7:30 UTC:

```
auto tp = local_days{2016y/may/29d} + 7h + 30min + 6s + 153ms; // 2016-05-29 07:30:06.153
auto zt = make_zoned("Asia/Tokyo", tp);
cout << zt << '\n';                                            // 2016-05-29 07:30:06.153 JST
```

The *only* change to the code is the use of `local_days` in place of `sys_days`. `local_days` is also a `std::chrono::time_point` but its "clock type" `local_t` has no `now()` function. This `time_point` is called `local_time`. A `local_time` can refer to *any* time zone. In the above example when we pair "Asia/Tokyo" with the `local_time`, the result becomes a `zoned_time` with the local time specified by the `local_time`.

To interoperate with time zones, calendrical types must convert to and from `local_days` as well as `sys_days`. The math is identical for both conversions, so it is very easy for the calendar author to provide. But as seen in this example, the meaning can be quite different.

The client of the calendar library can easily use the calendar types with the time zone library, specifying times either in the local time, or in UTC, simply by switching between `local_days` and `sys_days`. Here is an example that sets up a meeting at 9am on the third Tuesday of June, 2016 in New York:

```
auto zt = make_zoned("America/New_York", local_days{tue[3]/jun/2016} + 9h);
cout << zt << '\n';  // 2016-06-21 09:00:00 EDT
```

Need to set up a video conference with your partners in Helsinki?

```
cout << make_zoned("Europe/Helsinki", zt) << '\n';
```

This converts one `zoned_time` into another `zoned_time` where the only difference is changing from "America/New_York" to "Europe/Helsinki". The conversion preserves the UTC equivalent in both `zoned_time`s, and therefore outputs:

```
2016-06-21 16:00:00 EEST
```

And if this is not the formatting you prefer, that is easily fixed too:

```
cout << format("%F %H:%M %z", make_zoned("Europe/Helsinki", zt)) << '\n';
// 2016-06-21 16:00 +0300
```

Or perhaps properly localized:

```
cout << format(locale{"fi_FI"}, "%c", make_zoned("Europe/Helsinki", zt)) << '\n';
// Ti 21 Kes 16:00:00 2016
```

Wait, slow down, this is too much information! Let's start at the beginning. How do I get the current time?

```
cout << system_clock::now() << " UTC\n";
// 2016-05-30 17:57:30.694574 UTC
```

My current local time?

```
cout << make_zoned(current_zone(), system_clock::now()) << '\n';
// 2016-05-30 13:57:30.694574 EDT
```

Current time in Budapest?

```
cout << make_zoned("Europe/Budapest", system_clock::now()) << '\n';
// 2016-05-30 19:57:30.694574 CEST
```

For more documentation about the calendar portion of this proposal, including more details, more examples, and performance analyses, please see:

http://howardhinnant.github.io/date/date.html

For a video introduction to the calendar portion, please see:

https://www.youtube.com/watch?v=tzyGjOm8AKo

For a video introduction to the time zone portion, please see:

https://www.youtube.com/watch?v=Vwd3pduVGKY

For more documentation about the time zone portion of this proposal, including more details, and more examples, please see:

http://howardhinnant.github.io/date/tz.html

For more examples, some of which are written by users of this library, please see:

https://github.com/HowardHinnant/date/wiki/Examples-and-Recipes

For another example calendar which models the ISO week-based calendar, please see:

http://howardhinnant.github.io/date/iso_week.html

Issues

This is a collection of issues that could be changed one way or the other with this proposal.

1. Can the database be updated by the program while the program is running?

   This is probably the most important issue to be decided. This decision, one way or the other, leads (or doesn't) to many other decisions. If the database can be updated while the program is running:

   a. Is the cleint responsible for thread safety issues? This proposal says yes.
   b. What is the format of the database? This proposal says it is the text format of the IANA database.
   c. Can the database be updated to the latest version at a remote server on application startup? This proposal says yes. This means Egypt-like time-zone changes (3 days notice) can be handled seamlessly.

   Not allowing the database to be dynamically updated is by far the simpler solution. This proposal shows you what dynamic updating could look like. It is far easier to remove this feature from a proposal than to add it. This proposal is designed in such a way that it is trivial to remove this functionality.

2. Currently this library passes `time_zone`s around with `const time_zone*`. Each `time_zone` is a non-copyable `const` singleton in the application (much like a `type_info`). Passing them around by pointers allows syntax such as:

   ```
   auto tz = current_zone();
   cout << tz->name() << '\n';
   ```

   But source functions such as `current_zone` and `locate_zone` *never* return `nullptr`. So it has been suggested that the library traffic in `const time_zone&` instead. This would change the above code snippet to:

   ```
   auto& tz = current_zone();
   cout << tz.name() << '\n';
   ```

   Either solution is workable. And whichever we choose, the client can get the other with `*current_zone()` or `&current_zone()`. And whichever we choose, we *will* make the library API self-consistent so that things like the following work no matter what with this syntax:

   ```
   cout << make_zoned(current_zone(), system_clock::now()) << '\n';
   ```

We simply need to decide if the default style guide for passing `time_zone`s around is `const time_zone*` or `const time_zone&`. And yes, it is ok for a client to have a `const time_zone*` which equals `nullptr`. And no, the library never provides a `const time_zone*` which is equal to `nullptr`.

Proposed Wording

Text in grey boxes is not proposed wording.

Insert into synopsis in 20.17.2 Header `<chrono>` synopsis [time.syn]:

```
namespace std {
namespace chrono {

// ...

// duration stream insertion
template <class charT, class traits, class Rep, class Period>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os,
               const duration<Rep, Period>& d);

// ...
// convenience typedefs
// ...
using days   = duration<signed integer type of at least 25 bits, ratio_multiply<ratio<24>, hours::period>>;
using weeks  = duration<signed integer type of at least 22 bits, ratio_multiply<ratio<7>, days::period>>;
using years  = duration<signed integer type of at least 17 bits, ratio_multiply<ratio<146097, 400>, days::period>>;
using months = duration<signed integer type of at least 20 bits, ratio_divide<years::period, ratio<12>>>;

// ...
// clocks
// ...
class utc_clock;
class tai_clock;
class gps_clock;

template <class Duration>
    using sys_time = time_point<system_clock, Duration>;
using sys_seconds  = sys_time<seconds>;
using sys_days     = sys_time<days>;

template <class Duration>
    using utc_time = time_point<utc_clock, Duration>;
using utc_seconds  = utc_time<seconds>;

template <class Duration>
    using tai_time = time_point<tai_clock, Duration>;
using tai_seconds  = tai_time<seconds>;

template <class Duration>
    using gps_time = time_point<gps_clock, Duration>;
using gps_seconds  = gps_time<seconds>;

template <class Duration>
    sys_time<common_type_t<Duration, seconds>>
    to_sys_time(const utc_time<Duration>& t);
template <class Duration>
    sys_time<common_type_t<Duration, seconds>>
    to_sys_time(const tai_time<Duration>& t);
template <class Duration>
    sys_time<common_type_t<Duration, seconds>>
    to_sys_time(const gps_time<Duration>& t);

template <class Duration>
    utc_time<common_type_t<Duration, seconds>>
    to_utc_time(const sys_time<Duration>& t);
template <class Duration>
    utc_time<common_type_t<Duration, seconds>>
    to_utc_time(const tai_time<Duration>& t) noexcept;
template <class Duration>
    utc_time<common_type_t<Duration, seconds>>
    to_utc_time(const gps_time<Duration>& t) noexcept;

template <class Duration>
    tai_time<common_type_t<Duration, seconds>>
    to_tai_time(const sys_time<Duration>& t);
```

```
template <class Duration>
    tai_time<common_type_t<Duration, seconds>>
    to_tai_time(const utc_time<Duration>& t) noexcept;
template <class Duration>
    tai_time<common_type_t<Duration, seconds>>
    to_tai_time(const gps_time<Duration>& t) noexcept;

template <class Duration>
    gps_time<common_type_t<Duration, seconds>>
    to_gps_time(const sys_time<Duration>& t);
template <class Duration>
    gps_time<common_type_t<Duration, seconds>>
    to_gps_time(const utc_time<Duration>& t) noexcept;
template <class Duration>
    gps_time<common_type_t<Duration, seconds>>
    to_gps_time(const tai_time<Duration>& t) noexcept;

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const sys_time<Duration>& tp);
template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const sys_days& dp);
template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const utc_time<Duration>& t);
template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const tai_time<Duration>& t);
template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const gps_time<Duration>& t);

struct local_t {};
template <class Duration>
    using local_time = time_point<local_t, Duration>;
using local_seconds  = local_time<seconds>;
using local_days     = local_time<days>;

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const local_time<Duration>& tp);

// civil calendar

struct last_spec;

class day;

constexpr bool operator==(const day& x, const day& y) noexcept;
constexpr bool operator!=(const day& x, const day& y) noexcept;
constexpr bool operator< (const day& x, const day& y) noexcept;
constexpr bool operator> (const day& x, const day& y) noexcept;
constexpr bool operator<=(const day& x, const day& y) noexcept;
constexpr bool operator>=(const day& x, const day& y) noexcept;

constexpr day  operator+(const day&  x, const days& y) noexcept;
constexpr day  operator+(const days& x, const day&  y) noexcept;
constexpr day  operator-(const day&  x, const days& y) noexcept;
constexpr days operator-(const day&  x, const day&  y) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const day& d);

class month;

constexpr bool operator==(const month& x, const month& y) noexcept;
constexpr bool operator!=(const month& x, const month& y) noexcept;
constexpr bool operator< (const month& x, const month& y) noexcept;
constexpr bool operator> (const month& x, const month& y) noexcept;
constexpr bool operator<=(const month& x, const month& y) noexcept;
constexpr bool operator>=(const month& x, const month& y) noexcept;

constexpr month  operator+(const month&  x, const months& y) noexcept;
constexpr month  operator+(const months& x,  const month& y) noexcept;
constexpr month  operator-(const month&  x, const months& y) noexcept;
constexpr months operator-(const month&  x,  const month& y) noexcept;
```

```
template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const month& m);

class year;

constexpr bool operator==(const year& x, const year& y) noexcept;
constexpr bool operator!=(const year& x, const year& y) noexcept;
constexpr bool operator< (const year& x, const year& y) noexcept;
constexpr bool operator> (const year& x, const year& y) noexcept;
constexpr bool operator<=(const year& x, const year& y) noexcept;
constexpr bool operator>=(const year& x, const year& y) noexcept;

constexpr year  operator+(const year&  x, const years& y) noexcept;
constexpr year  operator+(const years& x, const year&  y) noexcept;
constexpr year  operator-(const year&  x, const years& y) noexcept;
constexpr years operator-(const year&  x, const year&  y) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const year& y);

class weekday;

constexpr bool operator==(const weekday& x, const weekday& y) noexcept;
constexpr bool operator!=(const weekday& x, const weekday& y) noexcept;

constexpr weekday operator+(const weekday& x, const days&    y) noexcept;
constexpr weekday operator+(const days&    x, const weekday& y) noexcept;
constexpr weekday operator-(const weekday& x, const days&    y) noexcept;
constexpr days    operator-(const weekday& x, const weekday& y) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const weekday& wd);

class weekday_indexed;

constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;
constexpr bool operator!=(const weekday_indexed& x, const weekday_indexed& y) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const weekday_indexed& wdi);

class weekday_last;

constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;
constexpr bool operator!=(const weekday_last& x, const weekday_last& y) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const weekday_last& wdl);

class month_day;

constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
constexpr bool operator!=(const month_day& x, const month_day& y) noexcept;
constexpr bool operator< (const month_day& x, const month_day& y) noexcept;
constexpr bool operator> (const month_day& x, const month_day& y) noexcept;
constexpr bool operator<=(const month_day& x, const month_day& y) noexcept;
constexpr bool operator>=(const month_day& x, const month_day& y) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const month_day& md);

class month_day_last;

constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator!=(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator< (const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator> (const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator<=(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator>=(const month_day_last& x, const month_day_last& y) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const month_day_last& mdl);
```

```
class month_weekday;

constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;
constexpr bool operator!=(const month_weekday& x, const month_weekday& y) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const month_weekday& mwd);

class month_weekday_last;

constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;
constexpr bool operator!=(const month_weekday_last& x, const month_weekday_last& y) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const month_weekday_last& mwdl);

class year_month;

constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
constexpr bool operator!=(const year_month& x, const year_month& y) noexcept;
constexpr bool operator< (const year_month& x, const year_month& y) noexcept;
constexpr bool operator> (const year_month& x, const year_month& y) noexcept;
constexpr bool operator<=(const year_month& x, const year_month& y) noexcept;
constexpr bool operator>=(const year_month& x, const year_month& y) noexcept;

constexpr year_month operator+(const year_month& ym, const months& dm) noexcept;
constexpr year_month operator+(const months& dm, const year_month& ym) noexcept;
constexpr year_month operator-(const year_month& ym, const months& dm) noexcept;
constexpr months operator-(const year_month& x, const year_month& y) noexcept;
constexpr year_month operator+(const year_month& ym, const years& dy) noexcept;
constexpr year_month operator+(const years& dy, const year_month& ym) noexcept;
constexpr year_month operator-(const year_month& ym, const years& dy) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const year_month& ym);

class year_month_day;

constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator!=(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator< (const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator> (const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator<=(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator>=(const year_month_day& x, const year_month_day& y) noexcept;

constexpr year_month_day operator+(const year_month_day& ymd, const months& dm) noexcept;
constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;
constexpr year_month_day operator+(const year_month_day& ymd, const years& dy) noexcept;
constexpr year_month_day operator+(const years& dy, const year_month_day& ymd) noexcept;
constexpr year_month_day operator-(const year_month_day& ymd, const months& dm) noexcept;
constexpr year_month_day operator-(const year_month_day& ymd, const years& dy) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const year_month_day& ymd);

class year_month_day_last;

constexpr bool operator==(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator!=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator< (const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator> (const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator<=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator>=(const year_month_day_last& x, const year_month_day_last& y) noexcept;

constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const months& dm) noexcept;
constexpr year_month_day_last operator+(const months& dm, const year_month_day_last& ymdl) noexcept;
constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const years& dy) noexcept;
constexpr year_month_day_last operator+(const years& dy, const year_month_day_last& ymdl) noexcept;
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const months& dm) noexcept;
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const years& dy) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const year_month_day_last& ymdl);
```

```
class year_month_weekday;

constexpr bool operator==(const year_month_weekday& x, const year_month_weekday& y) noexcept;
constexpr bool operator!=(const year_month_weekday& x, const year_month_weekday& y) noexcept;

constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const months& dm) noexcept;
constexpr year_month_weekday operator+(const months& dm, const year_month_weekday& ymwd) noexcept;
constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const years& dy) noexcept;
constexpr year_month_weekday operator+(const years& dy, const year_month_weekday& ymwd) noexcept;
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const months& dm) noexcept;
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const years& dy) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const year_month_weekday& ymwdi);

class year_month_weekday_last;

constexpr bool operator==(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;
constexpr bool operator!=(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;

constexpr year_month_weekday_last operator+(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
constexpr year_month_weekday_last operator+(const months& dm, const year_month_weekday_last& ymwdl) noexcept;
constexpr year_month_weekday_last operator+(const year_month_weekday_last& ymwdl, const years& dy) noexcept;
constexpr year_month_weekday_last operator+(const years& dy, const year_month_weekday_last& ymwdl) noexcept;
constexpr year_month_weekday_last operator-(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
constexpr year_month_weekday_last operator-(const year_month_weekday_last& ymwdl, const years& dy) noexcept;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const year_month_weekday_last& ymwdl);

// civil calendar conventional syntax operators
constexpr year_month operator/(const year& y, const month& m) noexcept;
constexpr year_month operator/(const year& y, int   m) noexcept;
constexpr month_day operator/(const month& m, const day& d) noexcept;
constexpr month_day operator/(const month& m, int d) noexcept;
constexpr month_day operator/(int m, const day& d) noexcept;
constexpr month_day operator/(const day& d, const month& m) noexcept;
constexpr month_day operator/(const day& d, int m) noexcept;
constexpr month_day_last operator/(const month& m, last_spec) noexcept;
constexpr month_day_last operator/(int m, last_spec) noexcept;
constexpr month_day_last operator/(last_spec, const month& m) noexcept;
constexpr month_day_last operator/(last_spec, int m) noexcept;
constexpr month_weekday operator/(const month& m, const weekday_indexed& wdi) noexcept;
constexpr month_weekday operator/(int m, const weekday_indexed& wdi) noexcept;
constexpr month_weekday operator/(const weekday_indexed& wdi, const month& m) noexcept;
constexpr month_weekday operator/(const weekday_indexed& wdi, int m) noexcept;
constexpr month_weekday_last operator/(const month& m, const weekday_last& wdl) noexcept;
constexpr month_weekday_last operator/(int m, const weekday_last& wdl) noexcept;
constexpr month_weekday_last operator/(const weekday_last& wdl, const month& m) noexcept;
constexpr month_weekday_last operator/(const weekday_last& wdl, int m) noexcept;
constexpr year_month_day operator/(const year_month& ym, const day& d) noexcept;
constexpr year_month_day operator/(const year_month& ym, int d) noexcept;
constexpr year_month_day operator/(const year& y, const month_day& md) noexcept;
constexpr year_month_day operator/(int y, const month_day& md) noexcept;
constexpr year_month_day operator/(const month_day& md, const year& y) noexcept;
constexpr year_month_day operator/(const month_day& md, int y) noexcept;
constexpr year_month_day_last operator/(const year_month& ym, last_spec) noexcept;
constexpr year_month_day_last operator/(const year& y, const month_day_last& mdl) noexcept;
constexpr year_month_day_last operator/(int y, const month_day_last& mdl) noexcept;
constexpr year_month_day_last operator/(const month_day_last& mdl, const year& y) noexcept;
constexpr year_month_day_last operator/(const month_day_last& mdl, int y) noexcept;
constexpr year_month_weekday operator/(const year_month& ym, const weekday_indexed& wdi) noexcept;
constexpr year_month_weekday operator/(const year& y, const month_weekday& mwd) noexcept;
constexpr year_month_weekday operator/(int y, const month_weekday& mwd) noexcept;
constexpr year_month_weekday operator/(const month_weekday& mwd, const year& y) noexcept;
constexpr year_month_weekday operator/(const month_weekday& mwd, int y) noexcept;
constexpr year_month_weekday_last operator/(const year_month& ym, const weekday_last& wdl) noexcept;
constexpr year_month_weekday_last operator/(const year& y, const month_weekday_last& mwdl) noexcept;
constexpr year_month_weekday_last operator/(int y, const month_weekday_last& mwdl) noexcept;
constexpr year_month_weekday_last operator/(const month_weekday_last& mwdl, const year& y) noexcept;
constexpr year_month_weekday_last operator/(const month_weekday_last& mwdl, int y) noexcept;

// time_of_day
template <class Duration> class time_of_day;
template <> class time_of_day<hours>;
template <> class time_of_day<minutes>;
```

```
template <> class time_of_day<seconds>;
template <class Rep, class Period> class time_of_day<duration<Rep, Period>>;

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const time_of_day<hours>& t);

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const time_of_day<minutes>& t);

template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const time_of_day<seconds>& t);

template<class charT, class traits, class Rep, class Period>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const time_of_day<duration<Rep, Period>>& t);

template <class Rep, class Period>
    constexpr time_of_day<duration<Rep, Period>>
    make_time(const duration<Rep, Period>& d);

// time zone database

struct tzdb;
const tzdb& get_tzdb();
const time_zone* locate_zone(const string& tz_name);
const time_zone* current_zone();

// Remote time zone database -- Needs discussion

const tzdb&  reload_tzdb();
string       remote_version();
bool         remote_download(const string& version);
bool         remote_install(const string& version);

// exception classes
class nonexistent_local_time;
class ambiguous_local_time;

struct sys_info;
template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const sys_info& si);

struct local_info;
template<class charT, class traits>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const local_info& li);

enum class choose {earliest, latest};
class time_zone;

bool operator==(const time_zone& x, const time_zone& y) noexcept;
bool operator!=(const time_zone& x, const time_zone& y) noexcept;

bool operator<(const time_zone& x, const time_zone& y) noexcept;
bool operator>(const time_zone& x, const time_zone& y) noexcept;
bool operator<=(const time_zone& x, const time_zone& y) noexcept;
bool operator>=(const time_zone& x, const time_zone& y) noexcept;

template <class Duration> class zoned_time;

using zoned_seconds = zoned_time<seconds>;

template <class Duration1, class Duration2>
    bool
    operator==(const zoned_time<Duration1>& x, const zoned_time<Duration2>& y);

template <class Duration1, class Duration2>
    bool
    operator!=(const zoned_time<Duration1>& x, const zoned_time<Duration2>& y);

template <class Duration>
    basic_ostream<class charT, class traits>&
    operator<<(basic_ostream<class charT, class traits>& os, const zoned_time<Duration>& t);

// make_zoned
```

```
template <class Duration>
    zoned_time<common_type_t<Duration, seconds>>
    make_zoned(const sys_time<Duration>& tp);

template <class Duration>
    zoned_time<common_type_t<Duration, seconds>>
    make_zoned(const time_zone* zone, const local_time<Duration>& tp);

template <class Duration>
    zoned_time<common_type_t<Duration, seconds>>
    make_zoned(const string& name, const local_time<Duration>& tp);

template <class Duration>
    zoned_time<common_type_t<Duration, seconds>>
    make_zoned(const time_zone* zone, const local_time<Duration>& tp, choose c);

template <class Duration>
    zoned_time<common_type_t<Duration, seconds>>
    make_zoned(const string& name, const local_time<Duration>& tp, choose c);

template <class Duration>
    zoned_time<common_type_t<Duration, seconds>>
    make_zoned(const time_zone* zone, const zoned_time<Duration>& zt);

template <class Duration>
    zoned_time<common_type_t<Duration, seconds>>
    make_zoned(const string& name, const zoned_time<Duration>& zt);

template <class Duration>
    zoned_time<common_type_t<Duration, seconds>>
    make_zoned(const time_zone* zone, const zoned_time<Duration>& zt, choose c);

template <class Duration>
    zoned_time<common_type_t<Duration, seconds>>
    make_zoned(const string& name, const zoned_time<Duration>& zt, choose c);

template <class Duration>
    zoned_time<common_type_t<Duration, seconds>>
    make_zoned(const time_zone* zone, const sys_time<Duration>& st);

template <class Duration>
    zoned_time<common_type_t<Duration, seconds>>
    make_zoned(const string& name, const sys_time<Duration>& st);

// to_stream

template <class CharT, class Traits, class Rep, class Period>
void
to_stream(basic_ostream<CharT, Traits>& os, const CharT* fmt, const duration<Rep, Period>& d);

template <class CharT, class Traits, class Duration>
void
to_stream(basic_ostream<CharT, Traits>& os, const CharT* fmt,
          const local_time<Duration>& tp, const string* abbrev = nullptr,
          const seconds* offset_sec = nullptr);

template <class CharT, class Traits, class Duration>
void
to_stream(basic_ostream<CharT, Traits>& os, const CharT* fmt, const sys_time<Duration>& tp);

template <class CharT, class Traits, class Duration>
void
to_stream(basic_ostream<CharT, Traits>& os, const CharT* fmt, const zoned_time<Duration>& tp);

// format duration

template <class charT, class traits, class Rep, class Period>
basic_string<charT, traits>
format(const locale& loc, const basic_string<charT, traits>& fmt, const duration<Rep, Period>& d);

template <class charT, class traits, class Rep, class Period>
basic_string<charT, traits>
format(const basic_string<charT, traits>& fmt, const duration<Rep, Period>& d);

template <class charT, class Rep, class Period>
basic_string<charT>
format(const locale& loc, const charT* fmt, const duration<Rep, Period>& d);
```

```
template <class charT, class Rep, class Period>
basic_string<charT>
format(const charT* fmt, const duration<Rep, Period>& d);

// format local_time

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const locale& loc, const basic_string<class charT, class traits>& format,
       const local_time<Duration>& tp);

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const basic_string<class charT, class traits>& format, const local_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const locale& loc, const charT* format, const local_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const charT* format, const local_time<Duration>& tp);

// format sys_time

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const locale& loc, const basic_string<class charT, class traits>& format,
       const sys_time<Duration>& tp);

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const basic_string<class charT, class traits>& format, const sys_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const locale& loc, const charT* format, const sys_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const charT* format, const sys_time<Duration>& tp);

// format zoned_time

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const locale& loc, const basic_string<class charT, class traits>& format,
       const zoned_time<Duration>& tp);

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const basic_string<class charT, class traits>& format, const zoned_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const locale& loc, const charT* format, const zoned_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const charT* format, const zoned_time<Duration>& tp);

// parse duration

template <class Rep, class Period, class CharT, class Traits>
unspecified
parse(const basic_string<CharT, Traits>& format, duration<Rep, Period>& d);

template <class Rep, class Period, class CharT>
unspecified
parse(const CharT* format, duration<Rep, Period>& d);

// parse sys_time

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, sys_time<Duration>& tp);

template <class Duration, class charT, class traits>
unspecified
```

```
parse(const basic_string<charT, traits>& format, sys_time<Duration>& tp,
      basic_string<charT, traits>& abbrev);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, sys_time<Duration>& tp,
      minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, sys_time<Duration>& tp,
      basic_string<charT, traits>& abbrev, minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, sys_time<Duration>& tp,
      minutes& offset, basic_string<charT, traits>& abbrev);

template <class Duration, class charT>
unspecified
parse(const charT* format, sys_time<Duration>& tp);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, sys_time<Duration>& tp,
      basic_string<charT, traits>& abbrev);

template <class Duration, class charT>
unspecified
parse(const charT* format, sys_time<Duration>& tp,
      minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, sys_time<Duration>& tp,
      basic_string<charT, traits>& abbrev, minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, sys_time<Duration>& tp,
      minutes& offset, basic_string<charT, traits>& abbrev);

// parse local_time

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, local_time<Duration>& tp);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, local_time<Duration>& tp,
      basic_string<charT, traits>& abbrev);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, local_time<Duration>& tp,
      minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, local_time<Duration>& tp,
      basic_string<charT, traits>& abbrev, minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, local_time<Duration>& tp,
      minutes& offset, basic_string<charT, traits>& abbrev);

template <class Duration, class charT>
unspecified
parse(const charT* format, local_time<Duration>& tp);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, local_time<Duration>& tp,
      basic_string<charT, traits>& abbrev);

template <class Duration, class charT>
unspecified
```

```
parse(const charT* format, local_time<Duration>& tp,
      minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, local_time<Duration>& tp,
      basic_string<charT, traits>& abbrev, minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, local_time<Duration>& tp,
      minutes& offset, basic_string<charT, traits>& abbrev);

// leap second support

class leap;

bool operator==(const leap& x, const leap& y);
bool operator!=(const leap& x, const leap& y);
bool operator< (const leap& x, const leap& y);
bool operator> (const leap& x, const leap& y);
bool operator<=(const leap& x, const leap& y);
bool operator>=(const leap& x, const leap& y);

template <class Duration> bool operator==(const const leap&         x, const sys_time<Duration>& y);
template <class Duration> bool operator==(const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator!=(const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator!=(const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator< (const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator< (const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator> (const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator> (const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator<=(const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator<=(const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator>=(const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator>=(const sys_time<Duration>& x, const leap&               y);

class link;

bool operator==(const link& x, const link& y);
bool operator!=(const link& x, const link& y);
bool operator< (const link& x, const link& y);
bool operator> (const link& x, const link& y);
bool operator<=(const link& x, const link& y);
bool operator>=(const link& x, const link& y);

} // namespace chrono

inline namespace literals {
inline namespace chrono_literals {
// ...
constexpr chrono::last_spec last{};

constexpr chrono::weekday sun{0};
constexpr chrono::weekday mon{1};
constexpr chrono::weekday tue{2};
constexpr chrono::weekday wed{3};
constexpr chrono::weekday thu{4};
constexpr chrono::weekday fri{5};
constexpr chrono::weekday sat{6};

constexpr chrono::month jan{1};
constexpr chrono::month feb{2};
constexpr chrono::month mar{3};
constexpr chrono::month apr{4};
constexpr chrono::month may{5};
constexpr chrono::month jun{6};
constexpr chrono::month jul{7};
constexpr chrono::month aug{8};
constexpr chrono::month sep{9};
constexpr chrono::month oct{10};
constexpr chrono::month nov{11};
constexpr chrono::month dec{12};

constexpr chrono::day  operator "" d(unsigned long long d) noexcept;
constexpr chrono::year operator "" y(unsigned long long y) noexcept;
}
}
```

Add new section [time.duration.io] after 20.17.5.9 duration algorithms [time.duration.alg]:

20.17.5.10 duration stream insertion [time.duration.io]

```
template <class charT, class traits, class Rep, class Period>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os,
           const duration<Rep, Period>& d);
```

Effects: Equivalent to:

```
os << d.count() << get_units<charT>(typename Period::type{});
```

Where `get_units<charT>(typename Period::type{})` is an exposition-only function which returns a null-terminated string of `charT` which depends on `Period::type` as follows (let `period` be the type `Period::type`):

- If `period` is type `atto`, `as`, else
- if `period` is type `femto`, `fs`, else
- if `period` is type `pico`, `ps`, else
- if `period` is type `nano`, `ns`, else
- if `period` is type `micro`, `μs` (U+00B5), else
- if `period` is type `milli`, `ms`, else
- if `period` is type `centi`, `cs`, else
- if `period` is type `deci`, `ds`, else
- if `period` is type `ratio<1>`, `s`, else
- if `period` is type `deca`, `das`, else
- if `period` is type `hecto`, `hs`, else
- if `period` is type `kilo`, `ks`, else
- if `period` is type `mega`, `Ms`, else
- if `period` is type `giga`, `Gs`, else
- if `period` is type `tera`, `Ts`, else
- if `period` is type `peta`, `Ps`, else
- if `period` is type `exa`, `Es`, else
- if `period` is type `ratio<60>`, `min`, else
- if `period` is type `ratio<3600>`, `h`, else
- if `period::den == 1`, `[num]s`, else
- `[num/den]s`.

In the list above the use of `num` and `den` refer to the static data members of `period` which are converted to arrays of `charT` using a decimal conversion with no leading zeroes.

For streams with `charT` which has a representation of 8 bits `μs` should be encoded as UTF-8. Otherwise UTF-16 or UTF-32 is encouraged. The implementation may substitute other encodings, including `us`.

Returns: `os`.

Add to synopsis in section [time.point] 20.17.6 Class template `time_point`:

```
template <class Clock, class Duration = typename Clock::duration>
class time_point {
public:
...
// 20.17.6.3, arithmetic
constexpr time_point& operator++();
constexpr time_point operator++(int);
constexpr time_point& operator--();
constexpr time_point operator--(int);

constexpr time_point& operator+=(const duration& d);
constexpr time_point& operator-=(const duration& d);
...
};
```

Add to synopsis in section [time.point.arithmetic] 20.17.6.3 `time_point` arithmetic:

```
constexpr time_point& operator++();
```

Effects: `++d_`.

Returns: `*this`.

```
constexpr time_point operator++(int);
```

Returns: `time_point{d_++}`.

```
constexpr time_point& operator--();
```

Effects: `--d_`.

Returns: `*this`.

```
constexpr time_point operator--(int);
```

Returns: `time_point{d_--}`.

Modify 20.17.7 [time.clock]:

1 The types defined in this subclause shall satisfy the TrivialClock requirements (20.17.3) unless otherwise sepcified.

Modify 20.17.7.1 [time.clock.system]:

1 Objects of class `system_clock` represent wall clock time from the system-wide realtime clock. `sys_time<Duration>` measures time since (and before) 1970-01-01 00:00:00 UTC *excluding* leap seconds. This measure is commonly referred to as *Unix Time*. This measure facilitates an efficient mapping between `sys_time` and calendar types ([time.calendar])

[*Example:*

```
sys_seconds{sys_days{1970y/jan/1}}.time_since_epoch() is 0s
sys_seconds{sys_days{2000y/jan/1}}.time_since_epoch() is 946'684'800s which is 10'957 * 86'400s
```

—end example]

Append new paragraphs after 20.17.7.1 [time.clock.system]/p4:

```
template <class Duration>
sys_time<common_type_t<Duration, seconds>>
to_sys_time(const utc_time<Duration>& u);
```

Returns: A `sys_time t`, such that `to_utc_time(t) == u` if such a mapping exists. Otherwise `u` represents a time_point during a leap second insertion and the last representable value of `sys_time prior` to the insertion of the leap second is returned.

[*Example:*

```
auto t = sys_days{jul/1/2015} - 500ms;
auto u = utc_clock::sys_to_utc(t);
t = to_sys_time(u);
assert(u.time_since_epoch() - t.time_since_epoch() == 25s);
cout << t << " SYS == " << u << " UTC\n";
u += 250ms;
t = to_sys_time(u);
assert(u.time_since_epoch() - t.time_since_epoch() == 25s);
cout << t << " SYS == " << u << " UTC\n";
u += 250ms;
t = to_sys_time(u);
assert(u.time_since_epoch() - t.time_since_epoch() == 25001ms);
cout << t << " SYS == " << u << " UTC\n";
u += 250ms;
t = to_sys_time(u);
assert(u.time_since_epoch() - t.time_since_epoch() == 25251ms);
cout << t << " SYS == " << u << " UTC\n";
u += 250ms;
t = to_sys_time(u);
assert(u.time_since_epoch() - t.time_since_epoch() == 25501ms);
cout << t << " SYS == " << u << " UTC\n";
u += 250ms;
t = to_sys_time(u);
assert(u.time_since_epoch() - t.time_since_epoch() == 25751ms);
cout << t << " SYS == " << u << " UTC\n";
u += 250ms;
t = to_sys_time(u);
assert(u.time_since_epoch() - t.time_since_epoch() == 26s);
cout << t << " SYS == " << u << " UTC\n";
u += 250ms;
t = to_sys_time(u);
assert(u.time_since_epoch() - t.time_since_epoch() == 26s);
cout << t << " SYS == " << u << " UTC\n";
```

Output:

```
2015-06-30 23:59:59.500 SYS == 2015-06-30 23:59:59.500 UTC
2015-06-30 23:59:59.750 SYS == 2015-06-30 23:59:59.750 UTC
2015-06-30 23:59:59.999 SYS == 2015-06-30 23:59:60.000 UTC
2015-06-30 23:59:59.999 SYS == 2015-06-30 23:59:60.250 UTC
2015-06-30 23:59:59.999 SYS == 2015-06-30 23:59:60.500 UTC
2015-06-30 23:59:59.999 SYS == 2015-06-30 23:59:60.750 UTC
2015-07-01 00:00:00.000 SYS == 2015-07-01 00:00:00.000 UTC
2015-07-01 00:00:00.250 SYS == 2015-07-01 00:00:00.250 UTC
```

— *end example*]

```
template <class Duration>
sys_time<common_type_t<Duration, seconds>>
to_sys_time(const tai_time<Duration>& u);
```

Effects: Equivalent to: `return to_sys_time(to_utc_time(u));`

```
template <class Duration>
sys_time<common_type_t<Duration, seconds>>
to_sys_time(const gps_time<Duration>& u);
```

Effects: Equivalent to: `return to_sys_time(to_utc_time(u));`

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const sys_time<Duration>& tp);
```

Remarks: This operator shall not participate in overload resolution if `treat_as_floating_point<typename Duration::rep>::value` is true, or if `Duration{1} >= days{1}`.

Effects:

```
auto const dp = floor<days>(tp);
os << year_month_day{dp} << ' ' << make_time(tp-dp);
```

Returns: `os`.

[*Example:*

```
cout << sys_seconds{0s} << '\n';             // 1970-01-01 00:00:00
cout << sys_seconds{946'684'800s} << '\n';   // 2000-01-01 00:00:00
```

— *end example:*]

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const sys_days& dp);
```

Effects:

```
os << year_month_day{dp};
```

Returns: `os`.

Add new section [time.clock.utc] after 20.17.7.1 Class system_clock [time.clock.system]:

20.17.7.2 Class utc_clock [time.clock.utc]

```
class utc_clock
{
public:
    using duration                  = system_clock::duration;
    using rep                       = duration::rep;
    using period                    = duration::period;
    using time_point                = chrono::time_point<utc_clock>;
    static constexpr bool is_steady = unspecified;

    static time_point now();
};
```

In contrast to `sys_time` which does not take leap seconds into account, `utc_clock` and its associated `time_point`, `utc_time`, counts time, *including* leap seconds, since 1970-01-01 00:00:00 UTC.

[*Example:*

```
to_utc_time(sys_seconds{sys_days{1970y/jan/1}}).time_since_epoch() is 0s
to_utc_time(sys_seconds{sys_days{2000y/jan/1}}).time_since_epoch() is 946'684'822s which is 10'957 * 86'400s + 22s
```

—*end example*]

`utc_clock` is not a `TrivialClock` unless the implementation can guarantee that `utc_clock::now()` does not propagate an exception. [*Note:* `noexcept(to_utc_time(system_clock::now()))` is `false`. — *end note*]

```
static utc_clock::time_point utc_clock::now();
```

Returns: The implementations should supply the best measure available. This may be approximated with `to_utc_time(system_clock::now())`.

```
template <class Duration>
utc_time<common_type_t<Duration, seconds>>
to_utc_time(const sys_time<Duration>& t);
```

Returns: A `utc_time` `u`, such that `u.time_since_epoch() - t.time_since_epoch()` is equal to the number of leap seconds that were inserted between `t` and 1970-01-01. If `t` is exactly the date of leap second insertion, then the conversion counts that leap second as inserted.

[*Example:*

```
auto t = sys_days{jul/1/2015} - 2ns;
auto u = to_utc_time(t);
assert(u.time_since_epoch() - t.time_since_epoch() == 25s);
t += 1ns;
u = to_utc_time(t);
assert(u.time_since_epoch() - t.time_since_epoch() == 25s);
t += 1ns;
u = to_utc_time(t);
assert(u.time_since_epoch() - t.time_since_epoch() == 26s);
t += 1ns;
u = to_utc_time(t);
assert(u.time_since_epoch() - t.time_since_epoch() == 26s);
```

— *end example*]

```
template <class Duration>
utc_time<common_type_t<Duration, seconds>>
to_utc_time(const tai_time<Duration>& t) noexcept;
```

Returns: `utc_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} - 378691210s`

Note: `378691210s == sys_days{1970y/jan/1} - sys_days{1958y/jan/1} + 10s`

```
template <class Duration>
utc_time<common_type_t<Duration, seconds>>
to_utc_time(const gps_time<Duration>& t) noexcept;
```

Returns: `utc_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} + 315964809s`

Note: `315964809s == sys_days{1980y/jan/sun[1]} - sys_days{1970y/jan/1} + 9s`

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const utc_time<Duration>& t);
```

Effects: Streams to `os` the same value as it would for `to_sys_time(t)`, except that during a leap second insertion, the seconds field is streamed as `60` (plus whatever fractional seconds is applicable).

Returns: `os`.

[*Example:*

```
auto t = sys_days{jul/1/2015} - 500ms;
auto u = utc_clock::sys_to_utc(t);
for (auto i = 0; i < 8; ++i, u += 250ms)
    cout << u << " UTC\n";
```

Output:

```
2015-06-30 23:59:59.500 UTC
2015-06-30 23:59:59.750 UTC
2015-06-30 23:59:60.000 UTC
2015-06-30 23:59:60.250 UTC
```

```
2015-06-30 23:59:60.500 UTC
2015-06-30 23:59:60.750 UTC
2015-07-01 00:00:00.000 UTC
2015-07-01 00:00:00.250 UTC
```

— *end example*]

Add new section [time.clock.tai] after 20.17.7.2 Class utc_clock [time.clock.utc]:

20.17.7.3 Class tai_clock [time.clock.tai]

```
class tai_clock
{
public:
    using duration                  = system_clock::duration;
    using rep                       = duration::rep;
    using period                    = duration::period;
    using time_point                = chrono::time_point<tai_clock>;
    static constexpr bool is_steady = unspecified;

    static time_point now();
};
```

The clock `tai_clock` measures seconds since 1958-01-01 00:00:00 and is offset 10s ahead of UTC at this date. That is, 1958-01-01 00:00:00 TAI is equivalent to 1957-12-31 23:59:50 UTC. Leap seconds are not inserted into TAI. Therefore every time a leap second is inserted into UTC, UTC falls another second behind TAI. For example by 2000-01-01 there had been 22 leap seconds inserted so 2000-01-01 00:00:00 UTC is equivalent to 2000-01-01 00:00:32 TAI (22s plus the initial 10s offset).

`tai_clock` is not a `TrivialClock` unless the implementation can guarantee that `tai_clock::now()` does not propagate an exception. [*Note:* `noexcept(to_tai_time(system_clock::now()))` is `false`. — *end note*]

```
static tai_clock::time_point tai_clock::now();
```

Returns: The implementations should supply the best measure available. This may be approximated with `to_tai_time(system_clock::now())`.

```
template <class Duration>
tai_time<common_type_t<Duration, seconds>>
to_tai_time(const sys_time<Duration>& t);
```

Effects: Equivalent to: `return to_tai_time(to_utc_time(t));`.

```
template <class Duration>
tai_time<common_type_t<Duration, seconds>>
to_tai_time(const utc_time<Duration>& t) noexcept;
```

Returns: `tai_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} + 378691210s`

Note: `378691210s == sys_days{1970y/jan/1} - sys_days{1958y/jan/1} + 10s`

```
template <class Duration>
tai_time<common_type_t<Duration, seconds>>
to_tai_time(const gps_time<Duration>& t) noexcept;
```

Returns: `tai_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} + 694656019s`

Note: `694656019s == sys_days{1980y/jan/sun[1]} - sys_days{1958y/jan/1} + 19s`

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const tai_time<Duration>& t);
```

Effects: Equivalent to:

```
auto tp = sys_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} -
              (sys_days{1970y/jan/1} - sys_days{1958y/jan/1});
return os << tp;
```

Add new section [time.clock.gps] after 20.17.7.3 Class tai_clock [time.clock.tai]:

20.17.7.4 Class gps_clock [time.clock.gps]

```
class gps_clock
{
public:
```

```
    using duration                  = system_clock::duration;
    using rep                       = duration::rep;
    using period                    = duration::period;
    using time_point                = chrono::time_point<gps_clock>;
    static constexpr bool is_steady = unspecified;

    static time_point now();
};
```

The clock `gps_clock` measures seconds since The first Sunday of January, 1980 00:00:00 UTC. Leap seconds are not inserted into GPS. Therefore every time a leap second is inserted into UTC, UTC falls another second behind GPS. Aside from the offset from 1958y/jan/1 to 1980y/jan/sun[1] GPS is behind TAI by 19s due to the 10s offset between 1958 and 1970 and the additional 9 leap seconds inserted between 1970 and 1980.

`gps_clock` is not a `TrivialClock` unless the implementation can guarantee that `gps_clock::now()` does not propagate an exception. [*Note:* `noexcept(to_gps_time(system_clock::now()))` is `false`. — *end note*]

```
static gps_clock::time_point gps_clock::now();
```

Returns: The implementations should supply the best measure available. This may be approximated with `to_gps_time(system_clock::now())`.

```
template <class Duration>
gps_time<common_type_t<Duration, seconds>>
to_gps_time(const sys_time<Duration>& t);
```

Effects: Equivalent to: `return to_gps_time(to_utc_time(t));`.

```
template <class Duration>
gps_time<common_type_t<Duration, seconds>>
to_gps_time(const utc_time<Duration>& t) noexcept;
```

Returns: `gps_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} - 315964809s`

Note: `315964809s == sys_days{1980y/jan/sun[1]} - sys_days{1970y/jan/1} + 9s`

```
template <class Duration>
gps_time<common_type_t<Duration, seconds>>
to_gps_time(const tai_time<Duration>& t) noexcept;
```

Returns: `gps_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} - 694656019s`

Note: `694656019s == sys_days{1980y/jan/sun[1]} - sys_days{1958y/jan/1} + 19s`

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const gps_time<Duration>& t);
```

Effects: Equivalent to:

```
auto tp = sys_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} +
             (sys_days{1980y/jan/sun[1]} - sys_days{1970y/jan/1});
return os << tp;
```

Add new section [time.clock.local_time] after 20.17.7.3 Class high_resolution_clock [time.clock.hres]:

20.17.7.7 local_time [time.clock.local_time]

The family of time points denoted by `local_time<Duration>` are based on the *pseudo clock* `local_t`. `local_t` has no member `now()` and thus does not meet the clock requirements. Nevertheless `local_time<Duration>` serves the vital role of representing local time with respect to a not-yet-specified time zone. Aside from being able to get the current time, the complete `time_point` algebra is available for `local_time<Duration>` (just as for `sys_time<Duration>`).

The following stream insertion operators exist for `local_time<Duration>`:

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const local_time<Duration>& lt);
```

Effects:

```
os << sys_time<Duration>{lt.time_since_epoch()};
```

Returns: `os`.

Add new section [time.calendar] after 20.17.7 Clocks [time.clocks]:

20.17.8 The civil calendar [time.calendar]

The types in this subclause describe the civil (Gregorian) calendar and its relationship to `sys_days` and `local_days`.

20.17.8.1 Class `last_spec` [time.calendar.last]

The struct `last_spec` is used in conjunction with other calendar types to specify the last in a sequence. For example, depending on context, it can represent the last day of a month, or the last day of the week of a month.

There is an `constexpr` object of this type named `last` in the `chrono_literals` namespace.

```
struct last_spec
{
    explicit last_spec() = default;
};
```

20.17.8.2 Class `day` [time.calendar.day]

`day` represents a day of a month. It normally holds values in the range 1 to 31. However it may hold non-negative values outside this range. It can be constructed with any `unsigned` value, which will be subsequently truncated to fit into `day`'s unspecified internal storage. `day` is equality and less-than comparable, and participates in basic arithmetic with `days` representing the quantity between any two `day`'s. One can form a `day` literal with `d`. And one can stream out a `day` . `day` has explicit conversions to and from `unsigned`.

```
class day
{
    unsigned char d_;  // exposition only
public:
    day() = default;
    explicit constexpr day(unsigned d) noexcept;

    constexpr day& operator++()    noexcept;
    constexpr day  operator++(int) noexcept;
    constexpr day& operator--()    noexcept;
    constexpr day  operator--(int) noexcept;

    constexpr day& operator+=(const days& d) noexcept;
    constexpr day& operator-=(const days& d) noexcept;

    constexpr explicit operator unsigned() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const day& x, const day& y) noexcept;
constexpr bool operator!=(const day& x, const day& y) noexcept;
constexpr bool operator< (const day& x, const day& y) noexcept;
constexpr bool operator> (const day& x, const day& y) noexcept;
constexpr bool operator<=(const day& x, const day& y) noexcept;
constexpr bool operator>=(const day& x, const day& y) noexcept;

constexpr day  operator+(const day&  x, const days& y) noexcept;
constexpr day  operator+(const days& x, const day&  y) noexcept;
constexpr day  operator-(const day&  x, const days& y) noexcept;
constexpr days operator-(const day&  x, const day&  y) noexcept;

template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const day& d);
```

`day` is a trivially copyable class type.
`day` is a standard-layout class type.
`day` is a literal class type.

```
explicit constexpr day::day(unsigned d) noexcept;
```

Effects: Constructs an object of type `day` by constructing `d_` with `d`.

```
constexpr day& day::operator++() noexcept;
```

Effects: `++d_`.

Returns: `*this`.

```
constexpr day day::operator++(int) noexcept;
```

Effects: `++(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr day& day::operator--() noexcept;
```

Effects: `--d_`.

Returns: `*this`.

```
constexpr day day::operator--(int) noexcept;
```

Effects: `--(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr day& day::operator+=(const days& d) noexcept;
```

Effects: `*this = *this + d`.

Returns: `*this`.

```
constexpr day& day::operator-=(const days& d) noexcept;
```

Effects: `*this = *this - d`.

Returns: `*this`.

```
constexpr explicit day::operator unsigned() const noexcept;
```

Returns: `d_`.

```
constexpr bool day::ok() const noexcept;
```

Returns: `1 <= d_ && d_ <= 31`.

```
constexpr bool operator==(const day& x, const day& y) noexcept;
```

Returns: `unsigned{x} == unsigned{y}`.

```
constexpr bool operator!=(const day& x, const day& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const day& x, const day& y) noexcept;
```

Returns: `unsigned{x} < unsigned{y}`.

```
constexpr bool operator> (const day& x, const day& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const day& x, const day& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const day& x, const day& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr day  operator+(const day&  x, const days& y) noexcept;
```

Returns: `day{unsigned{x} + y.count()}`.

```
constexpr day  operator+(const days& x, const day&  y) noexcept;
```

Returns: `y + x`.

```
constexpr day  operator-(const day&  x, const days& y) noexcept;
```

Returns: `x + -y`.

```
constexpr days operator-(const day&  x, const day&  y) noexcept;
```

Returns: `days{static_cast<days::rep>(unsigned{x} - unsigned{y})}`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const day& d);
```

Effects: Inserts a decimal integral text representation of `d` into `os`. Single digit values are prefixed with `'0'`.

Returns: `os`.

```
constexpr day operator "" d(unsigned long long d) noexcept;
```

Returns: `day{static_cast<unsigned>(d)}`.

20.17.8.3 Class `month` [time.calendar.month]

`month` represents a month of a year. It normally holds values in the range 1 to 12. However it may hold non-negative values outside this range. It can be constructed with any `unsigned` value, which will be subsequently truncated to fit into `month`'s unspecified internal storage. `month` is equality and less-than comparable, and participates in basic arithmetic with `months` representing the quantity between any two `month`'s. One can stream out a `month`. `month` has explicit conversions to and from `unsigned`. There are 12 `month` constants, one for each month of the year in the `chrono_literals` namespace.

```
class month
{
    unsigned char m_;  // exposition only
public:
    month() = default;
    explicit constexpr month(unsigned m) noexcept;

    constexpr month& operator++()    noexcept;
    constexpr month  operator++(int) noexcept;
    constexpr month& operator--()    noexcept;
    constexpr month  operator--(int) noexcept;

    constexpr month& operator+=(const months& m) noexcept;
    constexpr month& operator-=(const months& m) noexcept;

    constexpr explicit operator unsigned() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const month& x, const month& y) noexcept;
constexpr bool operator!=(const month& x, const month& y) noexcept;
constexpr bool operator< (const month& x, const month& y) noexcept;
constexpr bool operator> (const month& x, const month& y) noexcept;
constexpr bool operator<=(const month& x, const month& y) noexcept;
constexpr bool operator>=(const month& x, const month& y) noexcept;

constexpr month  operator+(const month&  x, const months& y) noexcept;
constexpr month  operator+(const months& x,  const month& y) noexcept;
constexpr month  operator-(const month&  x, const months& y) noexcept;
constexpr months operator-(const month&  x,  const month& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month& m);
```

`month` is a trivially copyable class type.
`month` is a standard-layout class type.
`month` is a literal class type.

```
explicit constexpr month::month(unsigned m) noexcept;
```

Effects: Constructs an object of type `month` by constructing `m_` with `m`.

```
constexpr month& month::operator++() noexcept;
```

Effects: If `m_ < 12`, `++m_`. Otherwise sets `m_` to 1.

Returns: `*this`.

```
constexpr month month::operator++(int) noexcept;
```

Effects: `++(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr month& month::operator--() noexcept;
```

Effects: If `m_ > 1`, `--m_`. Otherwise sets `m_` to 12.

Returns: `*this`.

```
constexpr month month::operator--(int) noexcept;
```

Effects: `--(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr month& month::operator+=(const months& m) noexcept;
```

Effects: `*this = *this + m`.

Returns: `*this`.

```
constexpr month& month::operator-=(const months& m) noexcept;
```

Effects: `*this = *this - m`.

Returns: `*this`.

```
constexpr explicit month::operator unsigned() const noexcept;
```

Returns: `m_`.

```
constexpr bool month::ok() const noexcept;
```

Returns: `1 <= m_ && m_ <= 12`.

```
constexpr bool operator==(const month& x, const month& y) noexcept;
```

Returns: `unsigned{x} == unsigned{y}`.

```
constexpr bool operator!=(const month& x, const month& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const month& x, const month& y) noexcept;
```

Returns: `unsigned{x} < unsigned{y}`.

```
constexpr bool operator> (const month& x, const month& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const month& x, const month& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const month& x, const month& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr month  operator+(const month&  x, const months& y) noexcept;
```

Returns: A `month` for which `ok() == true` and is found as if by incrementing (or decrementing if `y < months{0}`) `x`, `y` times. If `month.ok() == false` prior to this operation, behaves as if `*this` is first brought into the range [1, 12] by modular arithmetic. [*Note:* For example `month{0}` becomes `month{12}`, and `month{13}` becomes `month{1}`. — *end note*]

Complexity: O(1) with respect to the value of `y`. [*Note:* Repeated increments or decrements is not a valid implementation. — *end note*]

Example: `feb + months{11} == jan`.

```
constexpr month  operator+(const months& x, const month&  y) noexcept;
```

Returns: `y + x`.

```
constexpr month  operator-(const month&  x, const months& y) noexcept;
```

Returns: `x + -y`.

```
constexpr months operator-(const month&  x, const month&  y) noexcept;
```

Requires: `x.ok() == true` and `y.ok() == true`.

Returns: A value of `months` in the range of `months{0}` to `months{11}` inclusive.

Remarks: The returned value `m` shall satisfy the equality: `y + m == x`.

Example: `jan - feb == months{11}` .

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month& m);
```

Effects: If `ok() == true` outputs the same string that would be output for the month field by `asctime`. Otherwise outputs `unsigned{m} << " is not a valid month"`.

Returns: `os`.

20.17.8.4 Class `year` [time.calendar.year]

`year` represents a year in the civil calendar. It shall represent values in the range `[min(), max()]`. It can be constructed with any `int` value, which will be subsequently truncated to fit into `year`'s internal unspecified storage. `year` is equality and less-than comparable, and participates in basic arithmetic with `years` representing the quantity between any two `year`'s. One can form a `year` literal with `y`. And one can stream out a `year`. `year` has explicit conversions to and from `int`.

```
class year
{
    short y_;  // exposition only
public:
    year() = default;
    explicit constexpr year(int y) noexcept;

    constexpr year& operator++()    noexcept;
    constexpr year  operator++(int) noexcept;
    constexpr year& operator--()    noexcept;
    constexpr year  operator--(int) noexcept;

    constexpr year& operator+=(const years& y) noexcept;
    constexpr year& operator-=(const years& y) noexcept;

    constexpr year operator+() const noexcept;
    constexpr year operator-() const noexcept;

    constexpr bool is_leap() const noexcept;

    constexpr explicit operator int() const noexcept;
    constexpr bool ok() const noexcept;

    static constexpr year min() noexcept;
    static constexpr year max() noexcept;
};

constexpr bool operator==(const year& x, const year& y) noexcept;
constexpr bool operator!=(const year& x, const year& y) noexcept;
constexpr bool operator< (const year& x, const year& y) noexcept;
constexpr bool operator> (const year& x, const year& y) noexcept;
constexpr bool operator<=(const year& x, const year& y) noexcept;
constexpr bool operator>=(const year& x, const year& y) noexcept;

constexpr year  operator+(const year&  x, const years& y) noexcept;
constexpr year  operator+(const years& x, const year&  y) noexcept;
constexpr year  operator-(const year&  x, const years& y) noexcept;
constexpr years operator-(const year&  x, const year&  y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year& y);
```

`year` is a trivially copyable class type.
`year` is a standard-layout class type.
`year` is a literal class type.

```
explicit constexpr year::year(int y) noexcept;
```

Effects: Constructs an object of type `year` by constructing `y_` with `y`.

```
constexpr year& year::operator++() noexcept;
```

Effects: `++y_`.

Returns: `*this`.

```
constexpr year year::operator++(int) noexcept;
```

Effects: `++(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr year& year::operator--() noexcept;
```

Effects: `--y_`.

Returns: `*this`.

```
constexpr year year::operator--(int) noexcept;
```

Effects: `--(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr year& year::operator+=(const years& y) noexcept;
```

Effects: `*this = *this + y`.

Returns: `*this`.

```
constexpr year& year::operator-=(const years& y) noexcept;
```

Effects: `*this = *this - y`.

Returns: `*this`.

```
constexpr year& year::operator+() const noexcept;
```

Returns: `*this`.

```
constexpr year& year::operator-() const noexcept;
```

Returns: `year{-y_}`.

```
constexpr bool year::is_leap() const noexcept;
```

Returns: `true` if `*this` represents a leap year, else returns `false`.

```
constexpr explicit year::operator int() const noexcept;
```

Returns: `y_`.

```
constexpr bool year::ok() const noexcept;
```

Returns: `true`.

```
static constexpr year year::min() noexcept;
```

Returns: `year{numeric_limits<decltype(y_)>::min()}`.

```
static constexpr year year::max() noexcept;
```

Returns: `year{numeric_limits<decltype(y_)>::max()}`.

```
constexpr bool operator==(const year& x, const year& y) noexcept;
```

Returns: `int{x} == int{y}`.

```
constexpr bool operator!=(const year& x, const year& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const year& x, const year& y) noexcept;
```

Returns: `int{x} < int{y}`.

```
constexpr bool operator> (const year& x, const year& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const year& x, const year& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const year& x, const year& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr year  operator+(const year&  x, const years& y) noexcept;
```

Returns: `year{int{x} + y.count()}`.

```
constexpr year  operator+(const years& x, const year&  y) noexcept;
```

Returns: `y + x`.

```
constexpr year  operator-(const year&  x, const years& y) noexcept;
```

Returns: `x + -y`.

```
constexpr years operator-(const year&  x, const year&  y) noexcept;
```

Returns: `years{int{x} - int{y}}`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year& y);
```

Effects: Inserts a signed decimal integral text representation of `y` into `os`. If the year is in the range [-999, 999], prefixes the year with `'0'` to four digits. If the year is negative, prefixes with `'-'`.

Returns: `os`.

```
constexpr year operator "" y(unsigned long long y) noexcept;
```

Returns: `year{static_cast<int>(y)}`.

20.17.8.5 Class `weekday` [time.calendar.weekday]

`weekday` represents a day of the week in the civil calendar. It normally holds values in the range 0 to 6, corresponding to Sunday through Saturday. However it may hold non-negative values outside this range. It can be constructed with any `unsigned` value, which will be subsequently truncated to fit into `weekday`'s unspecified internal storage. `weekday` is equality comparable. `weekday` is not less-than comparable because there is no universal consensus on which day is the first day of the week. This design chooses the encoding of 0 to 6 to represent Sunday through Saturday only because this is consistent with existing C and C++ practice. However `weekday`'s comparison and arithmetic operations treat the days of the week as a circular range, with no beginning and no end. One can stream out a `weekday`. `weekday` has explicit conversions to and from `unsigned`. There are 7 `weekday` constants, one for each day of the week in the `chrono_literals` namespace.

A `weekday` can be implicitly constructed from a `sys_days`. This is the computation that discovers the day of the week of an arbitrary date.

A `weekday` can be indexed with either `unsigned` or `last`. This produces new types which represent the first, second, third, fourth, fifth or last weekdays of a month.

```
class weekday
{
    unsigned char wd_;  // exposition only
public:
    weekday() = default;
    explicit constexpr weekday(unsigned wd) noexcept;
    constexpr weekday(const sys_days& dp) noexcept;
    constexpr explicit weekday(const local_days& dp) noexcept;

    constexpr weekday& operator++()    noexcept;
    constexpr weekday  operator++(int) noexcept;
    constexpr weekday& operator--()    noexcept;
    constexpr weekday  operator--(int) noexcept;

    constexpr weekday& operator+=(const days& d) noexcept;
    constexpr weekday& operator-=(const days& d) noexcept;
```

```
    constexpr explicit operator unsigned() const noexcept;
    constexpr bool ok() const noexcept;

    constexpr weekday_indexed operator[](unsigned index) const noexcept;
    constexpr weekday_last    operator[](last_spec) const noexcept;
};

constexpr bool operator==(const weekday& x, const weekday& y) noexcept;
constexpr bool operator!=(const weekday& x, const weekday& y) noexcept;

constexpr weekday operator+(const weekday& x, const days&    y) noexcept;
constexpr weekday operator+(const days&    x, const weekday& y) noexcept;
constexpr weekday operator-(const weekday& x, const days&    y) noexcept;
constexpr days    operator-(const weekday& x, const weekday& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday& wd);
```

`weekday` is a trivially copyable class type.
`weekday` is a standard-layout class type.
`weekday` is a literal class type.

```
explicit constexpr weekday::weekday(unsigned wd) noexcept;
```

Effects: Constructs an object of type `weekday` by constructing `wd_` with `wd`.

```
constexpr weekday(const sys_days& dp) noexcept;
```

Effects: Constructs an object of type `weekday` by computing what day of the week corresponds to the `sys_days dp`, and representing that day of the week in `wd_`.

Example: If `dp` represents 1970-01-01, the constructed `weekday` represents Thursday by storing 4 in `wd_`.

```
constexpr explicit weekday(const local_days& dp) noexcept;
```

Effects: Constructs an object of type `weekday` by computing what day of the week corresponds to the `local_days dp`, and representing that day of the week in `wd_`.

The value after construction shall be identical to that constructed from `sys_days{dp.time_since_epoch()}`.

```
constexpr weekday& weekday::operator++() noexcept;
```

Effects: If `wd_ != 6`, `++wd_`. Otherwise sets `wd_` to 0.

Returns: `*this`.

```
constexpr weekday weekday::operator++(int) noexcept;
```

Effects: `++(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr weekday& weekday::operator--() noexcept;
```

Effects: If `wd_ != 0`, `--wd_`. Otherwise sets `wd_` to 6.

Returns: `*this`.

```
constexpr weekday weekday::operator--(int) noexcept;
```

Effects: `--(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr weekday& weekday::operator+=(const days& d) noexcept;
```

Effects: `*this = *this + d`.

Returns: `*this`.

```
constexpr weekday& weekday::operator-=(const days& d) noexcept;
```

Effects: `*this = *this - d`.

Returns: `*this`.

```
constexpr explicit weekday::operator unsigned() const noexcept;
```

Returns: `wd_`.

```
constexpr bool weekday::ok() const noexcept;
```

Returns: `wd_ <= 6`.

```
constexpr weekday_indexed weekday::operator[](unsigned index) const noexcept;
```

Returns: `{*this, index}`.

```
constexpr weekday_last weekday::operator[](last_spec) const noexcept;
```

Returns: `weekday_last{*this}`.

```
constexpr bool operator==(const weekday& x, const weekday& y) noexcept;
```

Returns: `unsigned{x} == unsigned{y}`.

```
constexpr bool operator!=(const weekday& x, const weekday& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr weekday  operator+(const weekday&  x, const days& y) noexcept;
```

Returns: A `weekday` for which `ok() == true` and is found as if by incrementing (or decrementing if `y < days{0}`) `x`, `y` times. If `weekday.ok() == false` prior to this operation, behaves as if `*this` is first brought into the range [0, 6] by modular arithmetic. [*Note:* For example `weekday{7}` becomes `weekday{0}`. — *end note*]

Complexity: O(1) with respect to the value of `y`. [*Note:* Repeated increments or decrements is not a valid implementation. — *end note*]

Example: `mon + days{6} == sun`.

```
constexpr weekday  operator+(const days& x, const weekday&  y) noexcept;
```

Returns: `y + x`.

```
constexpr weekday  operator-(const weekday&  x, const days& y) noexcept;
```

Returns: `x + -y`.

```
constexpr days operator-(const weekday&  x, const weekday&  y) noexcept;
```

Requires: `x.ok() == true` and `y.ok() == true`.

Returns: A value of `days` in the range of `days{0}` to `days{6}` inclusive.

Remarks: The returned value `d` shall satisfy the equality: `y + d == x`.

Example: `sun - mon == days{6}`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const weekday& wd);
```

Effects: If `ok() == true` outputs the same string that would be output for the weekday field by `asctime`. Otherwise outputs `unsigned{wd} << " is not a valid weekday"`.

Returns: `os`.

20.17.8.6 Class `weekday_indexed` [time.calendar.weekday_indexed]

`weekday_indexed` represents a `weekday` and a small index in the range 1 to 5. This class is used to represent the first, second, third, fourth or fifth weekday of a month. It is most easily constructed by indexing a `weekday`.

[*Example:*

```
constexpr auto wdi = sun[2];  // wdi is the second Sunday of an as yet unspecified month
static_assert(wdi.weekday() == sun);
static_assert(wdi.index() == 2);
```

— *end example:*]

```
class weekday_indexed
{
    chrono::weekday  wd_;     // exposition only
    unsigned char    index_;  // exposition only

public:
    constexpr weekday_indexed(const chrono::weekday& wd, unsigned index) noexcept;

    constexpr chrono::weekday weekday() const noexcept;
    constexpr unsigned        index()   const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;
constexpr bool operator!=(const weekday_indexed& x, const weekday_indexed& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday_indexed& wdi);
```

`weekday_indexed` is a trivially copyable class type.
`weekday_indexed` is a standard-layout class type.
`weekday_indexed` is a literal class type.

```
constexpr weekday_indexed::weekday_indexed(const chrono::weekday& wd, unsigned index) noexcept;
```

Effects: Constructs an object of type `weekday_indexed` by constructing `wd_` with `wd` and `index_` with `index`.

```
constexpr weekday weekday_indexed::weekday() const noexcept;
```

Returns: `wd_`.

```
constexpr unsigned weekday_indexed::index() const noexcept;
```

Returns: `index_`.

```
constexpr bool weekday_indexed::ok() const noexcept;
```

Returns: `wd_.ok() && 1 <= index_ && index_ <= 5`.

```
constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;
```

Returns: `x.weekday() == y.weekday() && x.index() == y.index()`.

```
constexpr bool operator!=(const weekday_indexed& x, const weekday_indexed& y) noexcept;
```

Returns: `!(x == y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const weekday_indexed& wdi);
```

Effects: Equivalent to:

```
return os << wdi.weekday() << '[' << wdi.index() << ']';
```

20.17.8.7 Class `weekday_last` [time.calendar.weekday_last]

`weekday_last` represents the last `weekday` of a month. It is most easily constructed by indexing a `weekday` with `last`.

[*Example:*

```
constexpr auto wdl = sun[last];  // wdl is the last Sunday of an as yet unspecified month
static_assert(wdl.weekday() == sun);
```

— *end example:*]

```
class weekday_last
{
    chrono::weekday wd_;  // exposition only

public:
    explicit constexpr weekday_last(const chrono::weekday& wd) noexcept;

    constexpr chrono::weekday weekday() const noexcept;
```

```
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;
constexpr bool operator!=(const weekday_last& x, const weekday_last& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday_last& wdl);
```

`weekday_last` is a trivially copyable class type.
`weekday_last` is a standard-layout class type.
`weekday_last` is a literal class type.

```
explicit constexpr weekday_last::weekday_last(const chrono::weekday& wd) noexcept;
```

Effects: Constructs an object of type `weekday_last` by constructing `wd_` with `wd`.

```
constexpr weekday weekday_last::weekday() const noexcept;
```

Returns: `wd_`.

```
constexpr bool weekday_last::ok() const noexcept;
```

Returns: `wd_.ok()`.

```
constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;
```

Returns: `x.weekday() == y.weekday()`.

```
constexpr bool operator!=(const weekday_last& x, const weekday_last& y) noexcept;
```

Returns: `!(x == y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const weekday_last& wdl);
```

Effects: Equivalent to:

```
    return os << wdi.weekday() << "[last]";
```

20.17.8.8 Class `month_day` [time.calendar.month_day]

`month_day` represents a specific `day` of a specific `month`, but with an unspecified `year`. One can observe the different components. One can assign a new value. `month_day` is equality comparable and less-than comparable. One can stream out a `month_day`.

```
class month_day
{
    chrono::month m_;  // exposition only
    chrono::day   d_;  // exposition only

public:
    month_day() = default;
    constexpr month_day(const chrono::month& m, const chrono::day& d) noexcept;

    constexpr chrono::month month() const noexcept;
    constexpr chrono::day   day()   const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
constexpr bool operator!=(const month_day& x, const month_day& y) noexcept;
constexpr bool operator< (const month_day& x, const month_day& y) noexcept;
constexpr bool operator> (const month_day& x, const month_day& y) noexcept;
constexpr bool operator<=(const month_day& x, const month_day& y) noexcept;
constexpr bool operator>=(const month_day& x, const month_day& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_day& md);
```

`month_day` is a trivially copyable class type.
`month_day` is a standard-layout class type.
`month_day` is a literal class type.

```
constexpr month_day::month_day(const chrono::month& m, const chrono::day& d) noexcept;
```

Effects: Constructs an object of type `month_day` by constructing `m_` with `m`, and `d_` with `d`.

```
constexpr month month_day::month() const noexcept;
```

Returns: `m_`.

```
constexpr day month_day::day() const noexcept;
```

Returns: `d_`.

```
constexpr bool month_day::ok() const noexcept;
```

Returns: `true` if `m_.ok()` is true, and if `1d <= d_`, and if `d_ <=` the number of days in month `m_`. For `m_ == feb` the number of days is considered to be 29. Otherwise returns `false`.

```
constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
```

Returns: `x.month() == y.month() && x.day() == y.day()`

```
constexpr bool operator!=(const month_day& x, const month_day& y) noexcept;
```

Returns: `!(x == y)`

```
constexpr bool operator< (const month_day& x, const month_day& y) noexcept;
```

Returns: If `x.month() < y.month()` returns `true`. Else if `x.month() > y.month()` returns `false`. Else returns `x.day() < y.day()`.

```
constexpr bool operator> (const month_day& x, const month_day& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const month_day& x, const month_day& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const month_day& x, const month_day& y) noexcept;
```

Returns: `!(x < y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month_day& md);
```

Effects: Equivalent to:

```
return os << md.month() << '/' << md.day();
```

20.17.8.9 Class `month_day_last` [time.calendar.month_day_last]

`month_day_last` represents the last `day` of a `month`. It is most easily constructed using the expression `m/last` or `last/m`, where `m` is an expression with type `month`.

[*Example:*

```
constexpr auto mdl = feb/last;  // mdl is the last day of February of an as yet unspecified year
static_assert(mdl.month() == feb);
```

— end example:]

```
class month_day_last
{
    chrono::month m_;  // exposition only

public:
    constexpr explicit month_day_last(const chrono::month& m) noexcept;

    constexpr chrono::month month() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator!=(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator< (const month_day_last& x, const month_day_last& y) noexcept;
```

```
constexpr bool operator> (const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator<=(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator>=(const month_day_last& x, const month_day_last& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_day_last& mdl);
```

`month_day_last` is a trivially copyable class type.
`month_day_last` is a standard-layout class type.
`month_day_last` is a literal class type.

```
constexpr explicit month_day_last::month_day_last(const chrono::month& m) noexcept;
```

Effects: Constructs an object of type `month_day_last` by constructing `m_` with `m`.

```
constexpr month month_day_last::month() const noexcept;
```

Returns: `m_`.

```
constexpr bool month_day_last::ok() const noexcept;
```

Returns: `m_.ok()`.

```
constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `x.month() == y.month()`.

```
constexpr bool operator!=(const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `!(x == y)`

```
constexpr bool operator< (const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `x.month() < y.month()`.

```
constexpr bool operator> (const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `!(x < y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month_day_last& mdl);
```

Effects: Equivalent to:

```
    return os << mdl.month() << "/last";
```

20.17.8.10 Class `month_weekday` [time.calendar.month_weekday]

`month_weekday` represents the nth `weekday` of a `month`, of an as yet unspecified `year`. To do this the `month_weekday` stores a `month` and a `weekday_indexed`.

```
class month_weekday
{
    chrono::month           m_;    // exposition only
    chrono::weekday_indexed wdi_;  // exposition only
public:
    constexpr month_weekday(const chrono::month& m, const chrono::weekday_indexed& wdi) noexcept;

    constexpr chrono::month           month()           const noexcept;
    constexpr chrono::weekday_indexed weekday_indexed() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;
constexpr bool operator!=(const month_weekday& x, const month_weekday& y) noexcept;
```

```
template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_weekday& mwd);
```

`month_weekday` is a trivially copyable class type.
`month_weekday` is a standard-layout class type.
`month_weekday` is a literal class type.

```
constexpr month_weekday::month_weekday(const chrono::month& m, const chrono::weekday_indexed& wdi) noexcept;
```

Effects: Constructs an object of type `month_weekday` by constructing `m_` with `m`, and `wdi_` with `wdi`.

```
constexpr month month_weekday::month() const noexcept;
```

Returns: `m_`.

```
constexpr weekday_indexed month_weekday::weekday_indexed() const noexcept;
```

Returns: `wdi_`.

```
constexpr bool month_weekday::ok() const noexcept;
```

Returns: `m_.ok() && wdi_.ok()`.

```
constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;
```

Returns: `x.month() == y.month() && x.weekday_indexed() == y.weekday_indexed()`.

```
constexpr bool operator!=(const month_weekday& x, const month_weekday& y) noexcept;
```

Returns: `!(x == y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month_weekday& mwd);
```

Effects: Equivalent to:

```
    return os << mwd.month() << '/' << mwd.weekday_indexed();
```

20.17.8.11 Class `month_weekday_last` [time.calendar.month_weekday_last]

`month_weekday_last` represents the last `weekday` of a `month`, of an as yet unspecified `year`. To do this the `month_weekday_last` stores a `month` and a `weekday_last`.

```
class month_weekday_last
{
    chrono::month        m_;     // exposition only
    chrono::weekday_last wdl_;   // exposition only
public:
    constexpr month_weekday_last(const chrono::month& m,
                                 const chrono::weekday_last& wdl) noexcept;

    constexpr chrono::month        month()        const noexcept;
    constexpr chrono::weekday_last weekday_last() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;
constexpr bool operator!=(const month_weekday_last& x, const month_weekday_last& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_weekday_last& mwdl);
```

`month_weekday_last` is a trivially copyable class type.
`month_weekday_last` is a standard-layout class type.
`month_weekday_last` is a literal class type.

```
constexpr month_weekday_last::month_weekday_last(const chrono::month& m,
                                                 const chrono::weekday_last& wdl) noexcept;
```

Effects: Constructs an object of type `month_weekday_last` by constructing `m_` with `m`, and `wdl_` with `wdl`.

```
constexpr month month_weekday_last::month() const noexcept;
```

Returns: `m_`.

```
constexpr weekday_last month_weekday_last::weekday_last() const noexcept;
```

Returns: `wdl_`.

```
constexpr bool month_weekday_last::ok() const noexcept;
```

Returns: `m_.ok() && wdl_.ok()`.

```
constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;
```

Returns: `x.month() == y.month() && x.weekday_last() == y.weekday_last()`.

```
constexpr bool operator!=(const month_weekday_last& x, const month_weekday_last& y) noexcept;
```

Returns: `!(x == y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month_weekday_last& mwdl);
```

Effects: Equivalent to:

```
return os << mwdl.month() << '/' << mwdl.weekday_last();
```

20.17.8.12 Class `year_month` [time.calendar.year_month]

`year_month` represents a specific `month` of a specific `year`, but with an unspecified `day`. `year_month` is a field-based time point with a resolution of `months`. One can observe the different components. One can assign a new value. `year_month` is equality comparable and less-than comparable. One can stream out a `year_month`.

```
class year_month
{
    chrono::year  y_;  // exposition only
    chrono::month m_;  // exposition only

public:
    year_month() = default;
    constexpr year_month(const chrono::year& y, const chrono::month& m) noexcept;

    constexpr chrono::year  year()  const noexcept;
    constexpr chrono::month month() const noexcept;

    constexpr year_month& operator+=(const months& dm) noexcept;
    constexpr year_month& operator-=(const months& dm) noexcept;
    constexpr year_month& operator+=(const years& dy)  noexcept;
    constexpr year_month& operator-=(const years& dy)  noexcept;

    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
constexpr bool operator!=(const year_month& x, const year_month& y) noexcept;
constexpr bool operator< (const year_month& x, const year_month& y) noexcept;
constexpr bool operator> (const year_month& x, const year_month& y) noexcept;
constexpr bool operator<=(const year_month& x, const year_month& y) noexcept;
constexpr bool operator>=(const year_month& x, const year_month& y) noexcept;

constexpr year_month operator+(const year_month& ym, const months& dm) noexcept;
constexpr year_month operator+(const months& dm, const year_month& ym) noexcept;
constexpr year_month operator-(const year_month& ym, const months& dm) noexcept;
constexpr months operator-(const year_month& x, const year_month& y) noexcept;
constexpr year_month operator+(const year_month& ym, const years& dy) noexcept;
constexpr year_month operator+(const years& dy, const year_month& ym) noexcept;
constexpr year_month operator-(const year_month& ym, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month& ym);
```

`year_month` is a trivially copyable class type.
`year_month` is a standard-layout class type.
`year_month` is a literal class type.

```
constexpr year_month::year_month(const chrono::year& y, const chrono::month& m) noexcept;
```

Effects: Constructs an object of type `year_month` by constructing `y_` with `y`, and `m_` with `m`.

```
constexpr year year_month::year() const noexcept;
```

Returns: `y_`.

```
constexpr month year_month::month() const noexcept;
```

Returns: `m_`.

```
constexpr year_month& operator+=(const months& dm) noexcept;
```

Effects: `*this = *this + dm`.

Returns: `*this`.

```
constexpr year_month& operator-=(const months& dm) noexcept;
```

Effects: `*this = *this - dm`.

Returns: `*this`.

```
constexpr year_month& operator+=(const years& dy) noexcept;
```

Effects: `*this = *this + dy`.

Returns: `*this`.

```
constexpr year_month& operator-=(const years& dy) noexcept;
```

Effects: `*this = *this - dy`.

Returns: `*this`.

```
constexpr bool year_month::ok() const noexcept;
```

Returns: `y_.ok() && m_.ok()`.

```
constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month()`

```
constexpr bool operator!=(const year_month& x, const year_month& y) noexcept;
```

Returns: `!(x == y)`

```
constexpr bool operator< (const year_month& x, const year_month& y) noexcept;
```

Returns: If `x.year() < y.year()` returns `true`. Else if `x.year() > y.year()` returns `false`. Else returns `x.month() < y.month()`.

```
constexpr bool operator> (const year_month& x, const year_month& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const year_month& x, const year_month& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const year_month& x, const year_month& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr year_month operator+(const year_month& ym, const months& dm) noexcept;
```

Returns: A `year_month` value `z` such that `z - ym == dm`.

Complexity: O(1) with respect to the value of `dm`.

```
constexpr year_month operator+(const months& dm, const year_month& ym) noexcept;
```

Returns: `ym + dm`.

```
constexpr year_month operator-(const year_month& ym, const months& dm) noexcept;
```

Returns: `ym + -dm`.

```
constexpr months operator-(const year_month& x, const year_month& y) noexcept;
```

Returns: The number of `months` one must add to `y` to get `x`.

```
constexpr year_month operator+(const year_month& ym, const years& dy) noexcept;
```

Returns: `(ym.year() + dy) / ym.month()`.

```
constexpr year_month operator+(const years& dy, const year_month& ym) noexcept;
```

Returns: `ym + dy`.

```
constexpr year_month operator-(const year_month& ym, const years& dy) noexcept;
```

Returns: `ym + -dy`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month& ym);
```

Effects: Equivalent to:

```
    return os << ym.year() << '/' << ym.month();
```

20.17.8.13 Class `year_month_day` [time.calendar.year_month_day]

`year_month_day` represents a specific `year`, `month`, and `day`. `year_month_day` is a field-based time point with a resolution of `days`. One can observe each field. `year_month_day` supports `years` and `months` oriented arithmetic, but not `days` oriented arithmetic. For the latter, there is a conversion to `sys_days` which efficiently supports `days` oriented arithmetic. There is also a conversion *from* `sys_days`. `year_month_day` is equality and less-than comparable.

```
class year_month_day
{
    chrono::year  y_;  // exposition only
    chrono::month m_;  // exposition only
    chrono::day   d_;  // exposition only

public:
    year_month_day() = default;
    constexpr year_month_day(const chrono::year& y, const chrono::month& m, const chrono::day& d) noexcept;
    constexpr year_month_day(const year_month_day_last& ymdl) noexcept;
    constexpr year_month_day(const sys_days& dp) noexcept;
    constexpr explicit year_month_day(const local_days& dp) noexcept;

    constexpr year_month_day& operator+=(const months& m) noexcept;
    constexpr year_month_day& operator-=(const months& m) noexcept;
    constexpr year_month_day& operator+=(const years& y)  noexcept;
    constexpr year_month_day& operator-=(const years& y)  noexcept;

    constexpr chrono::year  year()  const noexcept;
    constexpr chrono::month month() const noexcept;
    constexpr chrono::day   day()   const noexcept;

    constexpr          operator sys_days()   const noexcept;
    constexpr explicit operator local_days() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator!=(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator< (const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator> (const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator<=(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator>=(const year_month_day& x, const year_month_day& y) noexcept;

constexpr year_month_day operator+(const year_month_day& ymd, const months& dm) noexcept;
constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;
constexpr year_month_day operator+(const year_month_day& ymd, const years& dy) noexcept;
constexpr year_month_day operator+(const years& dy, const year_month_day& ymd) noexcept;
constexpr year_month_day operator-(const year_month_day& ymd, const months& dm) noexcept;
constexpr year_month_day operator-(const year_month_day& ymd, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_day& ymd);
```

`year_month_day` is a trivially copyable class type.

`year_month_day` is a standard-layout class type.
`year_month_day` is a literal class type.

```
constexpr year_month_day::year_month_day(const chrono::year& y, const chrono::month& m, const chrono::day& d) noexcept;
```

Effects: Constructs an object of type `year_month_day` by constructing `y_` with `y`, `m_` with `m`, and, `d_` with `d`.

```
constexpr year_month_day::year_month_day(const year_month_day_last& ymdl) noexcept;
```

Effects: Constructs an object of type `year_month_day` by constructing `y_` with `ymdl.year()`, `m_` with `ymdl.month()`, and, `d_` with `ymdl.day()`.

Note: This conversion from `year_month_day_last` to `year_month_day` is more efficient than converting a `year_month_day_last` to a `sys_days`, and then converting that `sys_days` to a `year_month_day`.

```
constexpr year_month_day::year_month_day(const sys_days& dp) noexcept;
```

Effects: Constructs an object of type `year_month_day` which corresponds to the date represented by `dp`.

Remarks: For any value of `year_month_day`, `ymd`, for which `ymd.ok()` is `true`, this equality will also be `true`: `ymd == year_month_day{sys_days{ymd}}`.

```
constexpr explicit year_month_day::year_month_day(const local_days& dp) noexcept;
```

Effects: Constructs an object of type `year_month_day` which corresponds to the date represented by `dp`.

Remarks: Equivalent to constructing with `sys_days{dp.time_since_epoch()}`.

```
constexpr year_month_day& year_month_day::operator+=(const months& m) noexcept;
```

Effects: `*this = *this + m;`.

Returns: `*this`.

```
constexpr year_month_day& year_month_day::operator-=(const months& m) noexcept;
```

Effects: `*this = *this - m;`.

Returns: `*this`.

```
constexpr year_month_day& year_month_day::operator+=(const years& y) noexcept;
```

Effects: `*this = *this + y;`.

Returns: `*this`.

```
constexpr year_month_day& year_month_day::operator-=(const years& y) noexcept;
```

Effects: `*this = *this - y;`.

Returns: `*this`.

```
constexpr year year_month_day::year() const noexcept;
```

Returns: `y_`.

```
constexpr month year_month_day::month() const noexcept;
```

Returns: `m_`.

```
constexpr day year_month_day::day() const noexcept;
```

Returns: `d_`.

```
constexpr year_month_day::operator sys_days() const noexcept;
```

Requires: `ok() == true`.

Returns: A `sys_days` which represents the date represented by `*this`.

Remarks: A `sys_days` which is converted to a `year_month_day`, shall have the same value when converted back to a `sys_days`. The round trip conversion sequence shall be *loss-less*.

```
constexpr explicit year_month_day::operator local_days() const noexcept;
```

Requires: `ok() == true`.

Effects: Equivalent to:

```
return local_days{static_cast<sys_days>(*this).time_since_epoch()};
```

```
constexpr bool year_month_day::ok() const noexcept;
```

Returns: If `y_.ok()` is `true`, and `m_.ok()` is `true`, and `d_` is in the range `[1d, (y_/m_/last).day()]`, then returns `true`, else returns `false`.

```
constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month() && x.day() == y.day()`.

```
constexpr bool operator!=(const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: If `x.year() < y.year()`, returns `true`. Else if `x.year() > y.year()` returns `false`. Else if `x.month() < y.month()`, returns `true`. Else if `x.month() > y.month()`, returns `false`. Else returns `x.day() < y.day()`.

```
constexpr bool operator> (const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr year_month_day operator+(const year_month_day& ymd, const months& dm) noexcept;
```

Requires: `ymd.month().ok()` is `true`.

Returns: `(ymd.year() / ymd.month() + dm) / ymd.day()`.

Remarks: If `ymd.day()` is in the range `[1d, 28d]`, the resultant `year_month_day` shall return `true` from `ok()`.

```
constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;
```

Returns: `ymd + dm`.

```
constexpr year_month_day operator-(const year_month_day& ymd, const months& dm) noexcept;
```

Returns: `ymd + (-dm)`.

```
constexpr year_month_day operator+(const year_month_day& ymd, const years& dy) noexcept;
```

Returns: `(ymd.year() + dy) / ymd.month() / ymd.day()`.

Remarks: If `ymd.month()` is `feb` and `ymd.day()` is not in the range `[1d, 28d]`, the resultant `year_month_day` may return `false` from `ok()`.

```
constexpr year_month_day operator+(const years& dy, const year_month_day& ymd) noexcept;
```

Returns: `ymd + dy`.

```
constexpr year_month_day operator-(const year_month_day& ymd, const years& dy) noexcept;
```

Returns: `ymd + (-dy)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month_day& ymd);
```

Effects: Inserts `yyyy-mm-dd` where the number of indicated digits are prefixed with `'0'` if necessary.

Returns: `os`.

20.17.8.14 Class `year_month_day_last` [time.calendar.year_month_day_last]

`year_month_day_last` represents a specific `year`, `month`, and the last `day` of the `month`. `year_month_day_last` is a field-based time point with a resolution of `days`, except that it is restricted to pointing to the last day of a year and month. One can observe each field. The `day` field is computed on demand. `year_month_day_last` supports `years` and `months` oriented arithmetic, but not `days` oriented arithmetic. For the latter, there is a conversion to `sys_days` which efficiently supports `days` oriented arithmetic. `year_month_day_last` is equality and less-than comparable.

```
class year_month_day_last
{
    chrono::year           y_;    // exposition only
    chrono::month_day_last mdl_;  // exposition only

public:
    constexpr year_month_day_last(const chrono::year& y,
                                  const chrono::month_day_last& mdl) noexcept;

    constexpr year_month_day_last& operator+=(const months& m) noexcept;
    constexpr year_month_day_last& operator-=(const months& m) noexcept;
    constexpr year_month_day_last& operator+=(const years& y)  noexcept;
    constexpr year_month_day_last& operator-=(const years& y)  noexcept;

    constexpr chrono::year           year()           const noexcept;
    constexpr chrono::month          month()          const noexcept;
    constexpr chrono::month_day_last month_day_last() const noexcept;
    constexpr chrono::day            day()            const noexcept;

    constexpr          operator sys_days()   const noexcept;
    constexpr explicit operator local_days() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator!=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator< (const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator> (const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator<=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator>=(const year_month_day_last& x, const year_month_day_last& y) noexcept;

constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const months& dm) noexcept;
constexpr year_month_day_last operator+(const months& dm, const year_month_day_last& ymdl) noexcept;
constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const years& dy) noexcept;
constexpr year_month_day_last operator+(const years& dy, const year_month_day_last& ymdl) noexcept;
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const months& dm) noexcept;
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_day_last& ymdl);
```

`year_month_day_last` is a trivially copyable class type.
`year_month_day_last` is a standard-layout class type.
`year_month_day_last` is a literal class type.

```
constexpr year_month_day_last::year_month_day_last(const chrono::year& y,
                                                   const chrono::month_day_last& mdl) noexcept;
```

Effects: Constructs an object of type `year_month_day_last` by constructing `y_` with `y` and `mdl_` with `mdl`.

```
constexpr year_month_day_last& year_month_day_last::operator+=(const months& m) noexcept;
```

Effects: `*this = *this + m;`.

Returns: `*this`.

```
constexpr year_month_day_last& year_month_day_last::operator-=(const months& m) noexcept;
```

Effects: `*this = *this - m;`.

Returns: `*this`.

```
constexpr year_month_day_last& year_month_day_last::operator+=(const years& y) noexcept;
```

Effects: `*this = *this + y;`.

Returns: `*this`.

```
constexpr year_month_day_last& year_month_day_last::operator-=(const years& y) noexcept;
```

Effects: `*this = *this - y;`.

Returns: `*this`.

```
constexpr year year_month_day_last::year() const noexcept;
```

Returns: `y_`.

```
constexpr month year_month_day_last::month() const noexcept;
```

Returns: `mdl_.month()`.

```
constexpr month_day_last year_month_day_last::month_day_last() const noexcept;
```

Returns: `mdl_`.

```
constexpr day year_month_day_last::day() const noexcept;
```

Returns: A `day` representing the last day of the `year`, `month` pair represented by `*this`.

```
constexpr year_month_day_last::operator sys_days() const noexcept;
```

Requires: `ok() == true`.

Returns: A `sys_days` which represents the date represented by `*this`.

```
constexpr explicit year_month_day_last::operator local_days() const noexcept;
```

Requires: `ok() == true`.

Effects: Equivalent to:

```
return local_days{static_cast<sys_days>(*this).time_since_epoch()};
```

```
constexpr bool year_month_day_last::ok() const noexcept;
```

Returns: `y_.ok() && mdl_.ok()`.

```
constexpr bool operator==(const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: `x.year() == y.year() && x.month_day_last() == y.month_day_last()`.

```
constexpr bool operator!=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: If `x.year() < y.year()`, returns `true`. Else if `x.year() > y.year()` returns `false`. Else returns `x.month_day_last() < y.month_day_last()`.

```
constexpr bool operator> (const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const months& dm) noexcept;
```

Requires: `ymdl.ok()` is `true`.

Returns: `(ymdl.year() / ymdl.month() + dm) / last`.

Postconditions: The resultant `year_month_day_last` returns `true` from `ok()`.

Complexity: O(1) with respect to the value of `dm`.

```
constexpr year_month_day_last operator+(const months& dm, const year_month_day_last& ymdl) noexcept;
```

Returns: `ymdl + dm`.

```
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const months& dm) noexcept;
```

Returns: `ymdl + (-dm)`.

```
constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const years& dy) noexcept;
```

Returns: `{ymdl.year()+dy, ymdl.month_day_last()}`.

```
constexpr year_month_day_last operator+(const years& dy, const year_month_day_last& ymdl) noexcept;
```

Returns: `ymdl + dy`.

```
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const years& dy) noexcept;
```

Returns: `ymdl + (-dy)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month_day_last& ymdl);
```

Effects: Equivalent to:

```
    return os << ymdl.year() << '/' << ymdl.month_day_last();
```

20.17.8.15 Class `year_month_weekday` [time.calendar.year_month_weekday]

`year_month_weekday` represents a specific `year`, `month`, and nth `weekday` of the `month`. `year_month_weekday` is a field-based time point with a resolution of `days`. One can observe each field. `year_month_weekday` supports `years` and `months` oriented arithmetic, but not `days` oriented arithmetic. For the latter, there is a conversion to `sys_days` which efficiently supports `days` oriented arithmetic. `year_month_weekday` is equality comparable.

```
class year_month_weekday
{
    chrono::year            y_;    // exposition only
    chrono::month           m_;    // exposition only
    chrono::weekday_indexed wdi_;  // exposition only

public:
    year_month_weekday() = default;
    constexpr year_month_weekday(const chrono::year& y, const chrono::month& m,
                                 const chrono::weekday_indexed& wdi) noexcept;
    constexpr year_month_weekday(const sys_days& dp) noexcept;
    constexpr explicit year_month_weekday(const local_days& dp) noexcept;

    constexpr year_month_weekday& operator+=(const months& m) noexcept;
    constexpr year_month_weekday& operator-=(const months& m) noexcept;
    constexpr year_month_weekday& operator+=(const years& y)  noexcept;
    constexpr year_month_weekday& operator-=(const years& y)  noexcept;

    constexpr chrono::year            year()            const noexcept;
    constexpr chrono::month           month()           const noexcept;
    constexpr chrono::weekday         weekday()         const noexcept;
    constexpr unsigned                index()           const noexcept;
    constexpr chrono::weekday_indexed weekday_indexed() const noexcept;

    constexpr          operator sys_days()   const noexcept;
    constexpr explicit operator local_days() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const year_month_weekday& x, const year_month_weekday& y) noexcept;
constexpr bool operator!=(const year_month_weekday& x, const year_month_weekday& y) noexcept;

constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const months& dm) noexcept;
constexpr year_month_weekday operator+(const months& dm, const year_month_weekday& ymwd) noexcept;
constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const years& dy) noexcept;
constexpr year_month_weekday operator+(const years& dy, const year_month_weekday& ymwd) noexcept;
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const months& dm) noexcept;
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_weekday& ymwdi);
```

`year_month_weekday` is a trivially copyable class type.
`year_month_weekday` is a standard-layout class type.

`year_month_weekday` is a literal class type.

```
constexpr year_month_weekday::year_month_weekday(const chrono::year& y, const chrono::month& m,
                                                  const chrono::weekday_indexed& wdi) noexcept;
```

Effects: Constructs an object of type `year_month_weekday` by constructing `y_` with `y`, `m_` with `m`, and `wdi_` with `wdi`.

```
constexpr year_month_weekday(const sys_days& dp) noexcept;
```

Effects: Constructs an object of type `year_month_weekday` which corresponds to the date represented by `dp`.

Remarks: For any value of `year_month_weekday`, `ymdl`, for which `ymdl.ok()` is `true`, this equality will also be `true`: `ymdl == year_month_weekday{sys_days{ymdl}}`.

```
constexpr explicit year_month_weekday(const local_days& dp) noexcept;
```

Effects: Constructs an object of type `year_month_weekday` which corresponds to the date represented by `dp`.

Remarks: Equivalent to constructing with `sys_days{dp.time_since_epoch()}`.

```
constexpr year_month_weekday& year_month_weekday::operator+=(const months& m) noexcept;
```

Effects: `*this = *this + m;`.

Returns: `*this`.

```
constexpr year_month_weekday& year_month_weekday::operator-=(const months& m) noexcept;
```

Effects: `*this = *this - m;`.

Returns: `*this`.

```
constexpr year_month_weekday& year_month_weekday::operator+=(const years& y) noexcept;
```

Effects: `*this = *this + y;`.

Returns: `*this`.

```
constexpr year_month_weekday& year_month_weekday::operator-=(const years& y) noexcept;
```

Effects: `*this = *this - y;`.

Returns: `*this`.

```
constexpr year year_month_weekday::year() const noexcept;
```

Returns: `y_`.

```
constexpr month year_month_weekday::month() const noexcept;
```

Returns: `m_`.

```
constexpr weekday year_month_weekday::weekday() const noexcept;
```

Returns: `wdi_.weekday()`.

```
constexpr unsigned year_month_weekday::index() const noexcept;
```

Returns: `wdi_.index()`.

```
constexpr weekday_indexed year_month_weekday::weekday_indexed() const noexcept;
```

Returns: `wdi_`.

```
constexpr year_month_weekday::operator sys_days() const noexcept;
```

Requires: `ok() == true`.

Returns: A `sys_days` which represents the date represented by `*this`.

```
constexpr explicit year_month_weekday::operator local_days() const noexcept;
```

Requires: `ok() == true`.

Effects: Equivalent to:

```
        return local_days{static_cast<sys_days>(*this).time_since_epoch()};
```

```
constexpr bool year_month_weekday::ok() const noexcept;
```

Returns: If `y_.ok()` or `m_.ok()` or `wdi_.ok()` returns `false`, returns `false`. Else if `*this` represents a valid date, returns `true`, else returns `false`.

```
constexpr bool operator==(const year_month_weekday& x, const year_month_weekday& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month() && x.weekday_indexed() == y.weekday_indexed()`.

```
constexpr bool operator!=(const year_month_weekday& x, const year_month_weekday& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const months& dm) noexcept;
```

Requires: `ymwd.ok()` is `true`.

Returns: `(ymwd.year() / ymwd.month() + dm) / ymwd.weekday_indexed()`.

Postconditions: The resultant `year_month_weekday` returns `true` from `ok()`.

Complexity: O(1) with respect to the value of `dm`.

```
constexpr year_month_weekday operator+(const months& dm, const year_month_weekday& ymwd) noexcept;
```

Returns: `ymwd + dm`.

```
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const months& dm) noexcept;
```

Returns: `ymwd + (-dm)`.

```
constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const years& dy) noexcept;
```

Returns: `{ymwd.year()+dy, ymwd.month(), ymwd.weekday_indexed()}`.

```
constexpr year_month_weekday operator+(const years& dy, const year_month_weekday& ymwd) noexcept;
```

Returns: `ymwd + dm`.

```
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const years& dy) noexcept;
```

Returns: `ymwd + (-dm)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month_weekday& ymwd);
```

Effects: Equivalent to:

```
        return os << ymwdi.year() << '/' << ymwdi.month() << '/' << ymwdi.weekday_indexed();
```

20.17.8.16 Class `year_month_weekday_last` [time.calendar.year_month_weekday_last]

`year_month_weekday_last` represents a specific `year`, `month`, and last `weekday` of the `month`. `year_month_weekday_last` is a field-based time point with a resolution of `days`, except that it is restricted to pointing to the last weekday of a year and month. One can observe each field. `year_month_weekday_last` supports `years` and `months` oriented arithmetic, but not `days` oriented arithmetic. For the latter, there is a conversion to `sys_days` which efficiently supports `days` oriented arithmetic. `year_month_weekday_last` is equality comparable.

```
class year_month_weekday_last
{
    chrono::year         y_;    // exposition only
    chrono::month        m_;    // exposition only
    chrono::weekday_last wdl_;  // exposition only

public:
    constexpr year_month_weekday_last(const chrono::year& y, const chrono::month& m,
                                      const chrono::weekday_last& wdl) noexcept;

    constexpr year_month_weekday_last& operator+=(const months& m) noexcept;
    constexpr year_month_weekday_last& operator-=(const months& m) noexcept;
    constexpr year_month_weekday_last& operator+=(const years& y)  noexcept;
    constexpr year_month_weekday_last& operator-=(const years& y)  noexcept;
```

```
    constexpr chrono::year         year()         const noexcept;
    constexpr chrono::month        month()        const noexcept;
    constexpr chrono::weekday      weekday()      const noexcept;
    constexpr chrono::weekday_last weekday_last() const noexcept;

    constexpr          operator sys_days()   const noexcept;
    constexpr explicit operator local_days() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr
bool
operator==(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;

constexpr
bool
operator!=(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;

constexpr
year_month_weekday_last
operator+(const year_month_weekday_last& ymwdl, const months& dm) noexcept;

constexpr
year_month_weekday_last
operator+(const months& dm, const year_month_weekday_last& ymwdl) noexcept;

constexpr
year_month_weekday_last
operator+(const year_month_weekday_last& ymwdl, const years& dy) noexcept;

constexpr
year_month_weekday_last
operator+(const years& dy, const year_month_weekday_last& ymwdl) noexcept;

constexpr
year_month_weekday_last
operator-(const year_month_weekday_last& ymwdl, const months& dm) noexcept;

constexpr
year_month_weekday_last
operator-(const year_month_weekday_last& ymwdl, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_weekday_last& ymwdl);
```

`year_month_weekday_last` is a trivially copyable class type.

`year_month_weekday_last` is a standard-layout class type.

`year_month_weekday_last` is a literal class type.

```
constexpr year_month_weekday_last::year_month_weekday_last(const chrono::year& y, const chrono::month& m,
                                                           const chrono::weekday_last& wdl) noexcept;
```

Effects: Constructs an object of type `year_month_weekday_last` by constructing `y_` with `y`, `m_` with `m`, and `wdl_` with `wdl`.

```
constexpr year_month_weekday_last& year_month_weekday_last::operator+=(const months& m) noexcept;
```

Effects: `*this = *this + m;`.

Returns: `*this`.

```
constexpr year_month_weekday_last& year_month_weekday_last::operator-=(const months& m) noexcept;
```

Effects: `*this = *this - m;`.

Returns: `*this`.

```
constexpr year_month_weekday_last& year_month_weekday_last::operator+=(const years& y) noexcept;
```

Effects: `*this = *this + y;`.

Returns: `*this`.

```
constexpr year_month_weekday_last& year_month_weekday_last::operator-=(const years& y) noexcept;
```

Effects: `*this = *this - y;`.

Returns: `*this`.

```
constexpr year year_month_weekday_last::year() const noexcept;
```

Returns: `y_`.

```
constexpr month year_month_weekday_last::month() const noexcept;
```

Returns: `m_`.

```
constexpr weekday year_month_weekday_last::weekday() const noexcept;
```

Returns: `wdl_.weekday()`.

```
constexpr weekday_last year_month_weekday_last::weekday_last() const noexcept;
```

Returns: `wdl_`.

```
constexpr year_month_weekday_last::operator sys_days() const noexcept;
```

Requires: `ok() == true`.

Returns: A `sys_days` which represents the date represented by `*this`.

```
constexpr explicit year_month_weekday_last::operator local_days() const noexcept;
```

Requires: `ok() == true`.

Effects: Equivalent to:

```
return local_days{static_cast<sys_days>(*this).time_since_epoch()};
```

```
constexpr bool year_month_weekday_last::ok() const noexcept;
```

Returns: If `y_.ok() && m_.ok() && wdl_.ok()`.

```
constexpr bool operator==(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month() && x.weekday_last() == y.weekday_last()`.

```
constexpr bool operator!=(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr year_month_weekday_last operator+(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
```

Requires: `ymwdl.ok()` is `true`.

Returns: `(ymwdl.year() / ymwdl.month() + dm) / ymwdl.weekday_last()`.

Postconditions: The resultant `year_month_weekday_last` returns `true` from `ok()`.

Complexity: O(1) with respect to the value of `dm`.

```
constexpr year_month_weekday_last operator+(const months& dm, const year_month_weekday_last& ymwdl) noexcept;
```

Returns: `ymwdl + dm`.

```
constexpr year_month_weekday_last operator-(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
```

Returns: `ymwdl + (-dm)`.

```
constexpr year_month_weekday_last operator+(const year_month_weekday_last& ymwdl, const years& dy) noexcept;
```

Returns: `{ymwdl.year()+dy, ymwdl.month(), ymwdl.weekday_last()}`.

```
constexpr year_month_weekday_last operator+(const years& dy, const year_month_weekday_last& ymwdl) noexcept;
```

Returns: `ymwdl + dy`.

```
constexpr year_month_weekday_last operator-(const year_month_weekday_last& ymwdl, const years& dy) noexcept;
```

Returns: `ymwdl + (-dy)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month_weekday_last& ymwdl);
```

Effects: Equivalent to:

```
return os << ymwdl.year() << '/' << ymwdl.month() << '/' << ymwdl.weekday_last();
```

20.17.8.17 civil calendar conventional syntax operators [time.calendar.operators]

A set of overloaded `operator/()` provide a conventional syntax for the creation of civil calendar dates. The year, month and day ordering are accepted in any of the following 3 orders:

1. `y/m/d`
2. `m/d/y`
3. `d/m/y`

Anywhere a "day" is required one can also specify one of:

- `last`
- `weekday[i]`
- `weekday[last]`

Partial-date-types such as `year_month` and `month_day` can be created by simply not applying the second division operator for any of the three orders. For example:

```
year_month ym = 2015y/apr;
month_day md1 = apr/4;
month_day md2 = 4d/apr;
```

Everything not intended as above is ill-formed, with the notable exception of an expression that consists of nothing but `int`, which has type `int`.

```
auto a = 2015/4/4;       // a == int(125)
auto b = 2015y/4/4;      // b == year_month_day{year(2015), month(4), day(4)}
auto c = 2015y/4d/apr;   // error: invalid operands to binary expression ('chrono::year' and 'chrono::day')
auto d = 2015/apr/4;     // error: invalid operands to binary expression ('int' and 'const chrono::month')
```

`year_month`:

```
constexpr year_month operator/(const year& y, const month& m) noexcept;
```

Returns: `{y, m}`.

```
constexpr year_month operator/(const year& y, int   m) noexcept;
```

Returns: `y / month(m)`.

`month_day`:

```
constexpr month_day operator/(const month& m, const day& d) noexcept;
```

Returns: `{m, d}`.

```
constexpr month_day operator/(const month& m, int d) noexcept;
```

Returns: `m / day(d)`.

```
constexpr month_day operator/(int m, const day& d) noexcept;
```

Returns: `month(m) / d`.

```
constexpr month_day operator/(const day& d, const month& m) noexcept;
```

Returns: `m / d`.

```
constexpr month_day operator/(const day& d, int m) noexcept;
```

Returns: `month(m) / d`.

`month_day_last`:

```
constexpr month_day_last operator/(const month& m, last_spec) noexcept;
```

Returns: `month_day_last{m}`.

```
constexpr month_day_last operator/(int m, last_spec) noexcept;
```

Returns: `month(m) / last`.

```
constexpr month_day_last operator/(last_spec, const month& m) noexcept;
```

Returns: `m / last`.

```
constexpr month_day_last operator/(last_spec, int m) noexcept;
```

Returns: `month(m) / last`.

`month_weekday`:

```
constexpr month_weekday operator/(const month& m, const weekday_indexed& wdi) noexcept;
```

Returns: `{m, wdi}`.

```
constexpr month_weekday operator/(int m, const weekday_indexed& wdi) noexcept;
```

Returns: `month(m) / wdi`.

```
constexpr month_weekday operator/(const weekday_indexed& wdi, const month& m) noexcept;
```

Returns: `m / wdi`.

```
constexpr month_weekday operator/(const weekday_indexed& wdi, int m) noexcept;
```

Returns: `month(m) / wdi`.

`month_weekday_last`:

```
constexpr month_weekday_last operator/(const month& m, const weekday_last& wdl) noexcept;
```

Returns: `{m, wdl}`.

```
constexpr month_weekday_last operator/(int m, const weekday_last& wdl) noexcept;
```

Returns: `month(m) / wdl`.

```
constexpr month_weekday_last operator/(const weekday_last& wdl, const month& m) noexcept;
```

Returns: `m / wdl`.

```
constexpr month_weekday_last operator/(const weekday_last& wdl, int m) noexcept;
```

Returns: `month(m) / wdl`.

`year_month_day`:

```
constexpr year_month_day operator/(const year_month& ym, const day& d) noexcept;
```

Returns: `{ym.year(), ym.month(), d}`.

```
constexpr year_month_day operator/(const year_month& ym, int d) noexcept;
```

Returns: `ym / day(d)`.

```
constexpr year_month_day operator/(const year& y, const month_day& md) noexcept;
```

Returns: `y / md.month() / md.day()`.

```
constexpr year_month_day operator/(int y, const month_day& md) noexcept;
```

Returns: `year(y) / md`.

```
constexpr year_month_day operator/(const month_day& md, const year& y) noexcept;
```

Returns: `y / md`.

```
constexpr year_month_day operator/(const month_day& md, int y) noexcept;
```

Returns: `year(y) / md`.

`year_month_day_last`:

```
constexpr year_month_day_last operator/(const year_month& ym, last_spec) noexcept;
```

Returns: `{ym.year(), month_day_last{ym.month()}}`.

`constexpr year_month_day_last operator/(const year& y, const month_day_last& mdl) noexcept;`

Returns: `{y, mdl}`.

`constexpr year_month_day_last operator/(int y, const month_day_last& mdl) noexcept;`

Returns: `year(y) / mdl`.

`constexpr year_month_day_last operator/(const month_day_last& mdl, const year& y) noexcept;`

Returns: `y / mdl`.

`constexpr year_month_day_last operator/(const month_day_last& mdl, int y) noexcept;`

Returns: `year(y) / mdl`.

`year_month_weekday`:

`constexpr year_month_weekday operator/(const year_month& ym, const weekday_indexed& wdi) noexcept;`

Returns: `{ym.year(), ym.month(), wdi}`.

`constexpr year_month_weekday operator/(const year& y, const month_weekday& mwd) noexcept;`

Returns: `{y, mwd.month(), mwd.weekday_indexed()}`.

`constexpr year_month_weekday operator/(int y, const month_weekday& mwd) noexcept;`

Returns: `year(y) / mwd`.

`constexpr year_month_weekday operator/(const month_weekday& mwd, const year& y) noexcept;`

Returns: `y / mwd`.

`constexpr year_month_weekday operator/(const month_weekday& mwd, int y) noexcept;`

Returns: `year(y) / mwd`.

`year_month_weekday_last`:

`constexpr year_month_weekday_last operator/(const year_month& ym, const weekday_last& wdl) noexcept;`

Returns: `{ym.year(), ym.month(), wdl}`.

`constexpr year_month_weekday_last operator/(const year& y, const month_weekday_last& mwdl) noexcept;`

Returns: `{y, mwdl.month(), mwdl.weekday_last()}`.

`constexpr year_month_weekday_last operator/(int y, const month_weekday_last& mwdl) noexcept;`

Returns: `year(y) / mwdl`.

`constexpr year_month_weekday_last operator/(const month_weekday_last& mwdl, const year& y) noexcept;`

Returns: `y / mwdl`.

`constexpr year_month_weekday_last operator/(const month_weekday_last& mwdl, int y) noexcept;`

Returns: `year(y) / mwdl`.

Add new section [time.time_of_day] after 20.17.8 The civil calendar [time.calendar]:

20.17.9 Class `time_of_day` [time.time_of_day]

The `time_of_day` class breaks a `duration` which represents the time elapsed since midnight, into a "broken" down time such as hours:minutes:seconds. The `Duration` template parameter dictates the precision to which the time is broken down. This can vary from a course precision of hours to a very fine precision of nanoseconds. `time_of_day` is primarily a formatting tool.

`template <class Duration> class time_of_day;`

There are 4 specializations of `time_of_day` to handle four precisions:

1. `tempalte <> class time_of_day<hours>`

   This specialization handles hours since midnight.

2. `tempalte <> class time_of_day<minutes>`

   This specialization handles hours:minutes since midnight.

3. `tempalte <> class time_of_day<seconds>`

   This specialization handles hours:minutes:seconds since midnight.

4. `tempalte <class Rep, class Period> class time_of_day<duration<Rep, Period>>`

   This specialization is restricted to `Rep` types that are integral, and `Period`s that are not an integral number of seconds. Typical uses are with milliseconds, microseconds and nanoseconds. This specialization handles hours:minute:seconds.fractional_seconds since midnight.

Each specialization of `time_of_day` is a trivially copyable class type.
Each specialization of `time_of_day` is a standard-layout class type.
Each specialization of `time_of_day` is a literal class type.

```
tempalte <>
class time_of_day<hours>
{
public:
    using precision = hours;

    constexpr explicit time_of_day(hours since_midnight) noexcept;

    constexpr hours    hours() const noexcept;

    constexpr explicit operator  precision()   const noexcept;
    constexpr          precision to_duration() const noexcept;

    constexpr void make24() noexcept;
    constexpr void make12() noexcept;
};
```

```
constexpr explicit time_of_day<hours>::time_of_day(hours since_midnight) noexcept;
```

Effects: Constructs an object of type `time_of_day` in 24-hour format corresponding to `since_midnight` hours after 00:00:00.

Postconditions: `hours()` returns the integral number of hours `since_midnight` is after 00:00:00.

```
constexpr hours time_of_day<hours>::hours() const noexcept;
```

Returns: The stored hour of `*this`.

```
constexpr explicit time_of_day<hours>::operator precision() const noexcept;
```

Returns: The number of hours since midnight.

```
constexpr precision to_duration() const noexcept;
```

Returns: `precision{*this}`.

```
constexpr void time_of_day<hours>::make24() noexcept;
```

Effects: If `*this` is a 12-hour time, converts to a 24-hour time. Otherwise, no effects.

```
constexpr void time_of_day<hours>::make12() noexcept;
```

Effects: If `*this` is a 24-hour time, converts to a 12-hour time. Otherwise, no effects.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const time_of_day<hours>& t);
```

Effects: If `t` is a 24-hour time, outputs to `os` according to the `strftime` format: "%H00". "%H" will emit a leading 0 for hours less than 10. Else `t` is a 12-hour time, outputs to `os` according to the `strftime` format: "%I%p" according to the C locale, except that no leading zero is output for hours less than 10.

Returns: `os`.

Example:

```
0100  // 1 in the morning in 24-hour format
```

```
          1800  // 6 in the evening in 24-hour format
          1am   // 1 in the morning in 12-hour format
          6pm   // 6 in the evening in 12-hour format

tempalte <>
class time_of_day<minutes>
{
public:
    using precision = minutes;

    constexpr explicit time_of_day(minutes since_midnight) noexcept;

    constexpr hours    hours()   const noexcept;
    constexpr minutes  minutes() const noexcept;

    constexpr explicit operator precision()    const noexcept;
    constexpr          precision to_duration() const noexcept;

    void make24() noexcept;
    void make12() noexcept;
};
```

`constexpr explicit time_of_day<minutes>::time_of_day(minutes since_midnight) noexcept;`

Effects: Constructs an object of type `time_of_day` in 24-hour format corresponding to `since_midnight` minutes after 00:00:00.

Postconditions: `hours()` returns the integral number of hours `since_midnight` is after 00:00:00. `minutes()` returns the integral number of minutes `since_midnight` is after (00:00:00 + `hours()`).

`constexpr hours time_of_day<minutes>::hours() const noexcept;`

Returns: The stored hour of `*this`.

`constexpr minutes time_of_day<minutes>::minutes() const noexcept;`

Returns: The stored minute of `*this`.

`constexpr explicit time_of_day<minutes>::operator precision() const noexcept;`

Returns: The number of minutes since midnight.

`constexpr precision to_duration() const noexcept;`

Returns: `precision{*this}`.

`void time_of_day<minutes>::make24() noexcept;`

Effects: If `*this` is a 12-hour time, converts to a 24-hour time. Otherwise, no effects.

`void time_of_day<minutes>::make12() noexcept;`

Effects: If `*this` is a 24-hour time, converts to a 12-hour time. Otherwise, no effects.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const time_of_day<minutes>& t);
```

Effects: If `t` is a 24-hour time, outputs to `os` according to the `strftime` format: "%H:%M". "%H" will emit a leading 0 for hours less than 10. Else `t` is a 12-hour time, outputs to `os` according to the `strftime` format: "%I:%M%p" according to the C locale, except that no leading zero is output for hours less than 10.

Returns: `os`.

Example:

```
          01:08   // 1:08 in the morning in 24-hour format
          18:15   // 6:15 in the evening in 24-hour format
          1:08am  // 1:08 in the morning in 12-hour format
          6:15pm  // 6:15 in the evening in 12-hour format

tempalte <>
class time_of_day<seconds>
{
public:
    using precision = seconds;
```

```
    constexpr explicit time_of_day(seconds since_midnight) noexcept;

    constexpr hours    hours()   const noexcept;
    constexpr minutes  minutes() const noexcept;
    constexpr seconds  seconds() const noexcept;

    constexpr explicit operator  precision()   const noexcept;
    constexpr          precision to_duration() const noexcept;

    void make24() noexcept;
    void make12() noexcept;
};
```

```
constexpr explicit time_of_day<seconds>::time_of_day(seconds since_midnight) noexcept;
```

Effects: Constructs an object of type `time_of_day` in 24-hour format corresponding to `since_midnight` seconds after 00:00:00.

Postconditions: `hours()` returns the integral number of hours `since_midnight` is after 00:00:00. `minutes()` returns the integral number of minutes `since_midnight` is after (00:00:00 + `hours()`). `seconds()` returns the integral number of seconds `since_midnight` is after (00:00:00 + `hours()` + `minutes()`).

```
constexpr hours time_of_day<seconds>::hours() const noexcept;
```

Returns: The stored hour of `*this`.

```
constexpr minutes time_of_day<seconds>::minutes() const noexcept;
```

Returns: The stored minute of `*this`.

```
constexpr seconds time_of_day<seconds>::seconds() const noexcept;
```

Returns: The stored second of `*this`.

```
constexpr explicit time_of_day<seconds>::operator precision() const noexcept;
```

Returns: The number of seconds since midnight.

```
constexpr precision to_duration() const noexcept;
```

Returns: `precision{*this}`.

```
void time_of_day<seconds>::make24() noexcept;
```

Effects: If `*this` is a 12-hour time, converts to a 24-hour time. Otherwise, no effects.

```
void time_of_day<seconds>::make12() noexcept;
```

Effects: If `*this` is a 24-hour time, converts to a 12-hour time. Otherwise, no effects.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const time_of_day<seconds>& t);
```

Effects: If `t` is a 24-hour time, outputs to `os` according to the `strftime` format: "%H:%M%S". "%H" will emit a leading 0 for hours less than 10. Else `t` is a 12-hour time, outputs to `os` according to the `strftime` format: "%I:%M%S%p" according to the C locale, except that no leading zero is output for hours less than 10.

Returns: `os`.

Example:

```
     01:08:03   // 1:08:03 in the morning in 24-hour format
     18:15:45   // 6:15:45 in the evening in 24-hour format
     1:08:03am  // 1:08:03 in the morning in 12-hour format
     6:15:45pm  // 6:15:45 in the evening in 12-hour format
```

```
tempalte <class Rep, class Period>
class time_of_day<duration<Rep, Period>>
{
public:
    using precision = duration<Rep, Period>;

    constexpr explicit time_of_day(precision since_midnight) noexcept;

    constexpr hours     hours()      const noexcept;
    constexpr minutes   minutes()    const noexcept;
```

```
    constexpr seconds   seconds()    const noexcept;
    constexpr precision subseconds() const noexcept;

    constexpr explicit operator  precision()   const noexcept;
    constexpr          precision to_duration() const noexcept;

    void make24() noexcept;
    void make12() noexcept;
};
```

This specialization shall not exist unless `treat_as_floating_point<Rep>::value` is `false` and `duration<Rep, Period>` is not convertible to `seconds`.

```
constexpr explicit time_of_day<duration<Rep, Period>>::time_of_day(precision since_midnight) noexcept;
```

Effects: Constructs an object of type `time_of_day` in 24-hour format corresponding to `since_midnight precision` fractional seconds after 00:00:00.

Postconditions: `hours()` returns the integral number of hours `since_midnight` is after 00:00:00. `minutes()` returns the integral number of minutes `since_midnight` is after (00:00:00 + `hours()`). `seconds()` returns the integral number of seconds `since_midnight` is after (00:00:00 + `hours()` + `minutes()`). `subseconds()` returns the integral number of fractional precision seconds `since_midnight` is after (00:00:00 + `hours()` + `minutes()` + `seconds`).

```
constexpr hours time_of_day<duration<Rep, Period>>::hours() const noexcept;
```

Returns: The stored hour of `*this`.

```
constexpr minutes time_of_day<duration<Rep, Period>>::minutes() const noexcept;
```

Returns: The stored minute of `*this`.

```
constexpr seconds time_of_day<duration<Rep, Period>>::seconds() const noexcept;
```

Returns: The stored second of `*this`.

```
constexpr
duration<Rep, Period>
time_of_day<duration<Rep, Period>>::subseconds() const noexcept;
```

Returns: The stored subsecond of `*this`.

```
constexpr explicit time_of_day<duration<Rep, Period>>::operator precision() const noexcept;
```

Returns: The number of subseconds since midnight.

```
constexpr precision to_duration() const noexcept;
```

Returns: `precision{*this}`.

```
void time_of_day<duration<Rep, Period>>::make24() noexcept;
```

Effects: If `*this` is a 12-hour time, converts to a 24-hour time. Otherwise, no effects.

```
void time_of_day<duration<Rep, Period>>::make12() noexcept;
```

Effects: If `*this` is a 24-hour time, converts to a 12-hour time. Otherwise, no effects.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const time_of_day<duration<Rep, Period>>& t);
```

Effects: If `t` is a 24-hour time, outputs to `os` according to the `strftime` format: "%H:%M%S.%s". "%H" will emit a leading 0 for hours less than 10. "%s" is not a `strftime` code and represents the fractional seconds. Else `t` is a 12-hour time, outputs to `os` according to the `strftime` format: "%I:%M%S.%s%p" according to the C locale, except that no leading zero is output for hours less than 10.

Returns: `os`.

Example:

```
01:08:03.007   // 1:08:03.007 in the morning in 24-hour format (assuming millisecond precision)
18:15:45.123   // 6:15:45.123 in the evening in 24-hour format (assuming millisecond precision)
1:08:03.007am  // 1:08:03.007 in the morning in 12-hour format (assuming millisecond precision)
6:15:45.123pm  // 6:15:45.123 in the evening in 12-hour format (assuming millisecond precision)
```

20.17.9.1 Helper functions `make_time` [time.time_of_day.make_time]

These helper functions ease the construction of `time_of_day` objects by deducing the precision for the `time_of_day` return type from the argument to the helper function.

```
template <class Rep, class Period>
constexpr
time_of_day<duration<Rep, Period>>
make_time(const duration<Rep, Period>& d);
```

Returns: `time_of_day<duration<Rep, Period>>(d)`.

Add new section [time.timezone] after 20.17.9 Class `time_of_day` [time.time_of_day]:

20.17.10 Time Zones [time.timezone]

This clause creates an API which exposes the IANA Time Zone database, and interfaces with `sys_time` and `local_time`. By using only this interface, time zone support is provided not only to the civil calendar types, but also to other user-written calendars that interface with `sys_time` and `local_time`.

20.17.10.1 The time zone database [time.timezone.database]

The following data structure is the time zone database, and the following functions access it.

```
struct tzdb
{
    string            version;
    vector<time_zone> zones;
    vector<link>      links;
    vector<leap>      leaps;
};
```

The tzdb database is a singleton. And access to it is *read-only*, except for `reload_tzdb()` which re-initializes it. Each `vector` is sorted to enable fast lookup. You can iterate over and inspect this database.

```
const tzdb& get_tzdb();
```

Effects: If this is the first access to the database, will initialize the database.

Returns: A `const` reference to the database.

Thread Safety: It is safe to call this function from multiple threads at one time.

Throws: `runtime_error` if for any reason a reference can not be returned to a valid `tzdb`.

```
const time_zone* locate_zone(const string& tz_name);
```

Effects: Calls `get_tzdb()` which will initialize the timezone database if this is the first reference to the database.

Returns: If a `time_zone` is found for which `name() == tz_name`, returns a pointer to that `time_zone`. Otherwise if a `link` is found where `tz_name == link.name()`, then a pointer is returned to the `time_zone` for which `zone.name() == link.target()` [*Note:* A `link` is an alternative name for a `time_zone`. — *end note*]

Throws: Any exception propagated from `get_tzdb()`. If a `const time_zone*` can not be found as described in the *Returns* clause, throws a `runtime_error`. [*Note:* On non-exceptional return, the return value is *always* a pointer to a valid `time_zone`. — *end note*]

```
const time_zone* current_zone();
```

Effects: Calls `locate_zone()` which will initialize the timezone database if this is the first reference to the database.

Returns: A `const time_zone*` referring to the time zone which your computer has set as its local time zone.

Throws: Any exception propagated from `locate_zone()`. [*Note:* On non-exceptional return, the return value is *always* a pointer to a valid `time_zone`. — *end note*]

20.17.10.1.1 Remote time zone database support [time.timezone.database.remote]

This subsection is optional/seperable and needs further discussion in the LEWG. No other sections depend upon this subsection.

```
const tzdb& reload_tzdb();
```

Effects: This function first checks the latest version at the IANA website. If the IANA website is unavailable, or if the latest version is already installed, there are no effects. Otherwise, a new version is available. It is downloaded and installed, and then the program re-initializes the `tzdb` singleton from the new disk files.

Returns: A `const` reference to the database.

Thread Safety: This function is *not* thread safe. You must provide your own synchronization among threads accessing the time zone database to safely use this function. If this function re-initializes the database, all outstanding `const time_zone*` are invalidated (including those held within `zoned_time` objects). And afterwards, all outstanding `sys_info` may hold obsolete data.

Throws: `runtime_error` if for any reason a reference can not be returned to a valid `tzdb`.

```
string remote_version();
```

Returns: The latest database version number from the IANA website. If the IANA website can not be reached, or if it can be reached but the latest version number is unexpectedly not available, the empty string is returned.

Note: If non-empty, this can be compared with `get_tzdb().version` to discover if you have the latest database installed.

```
bool remote_download(const string& version);
```

Effects: If `version == remote_version()` this function will download the compressed tar file holding the latest time zone database from the IANA website. The tar file will be placed at an unspecified location.

Returns: `true` if the database was successfully downloaded, else `false`.

Thread safety: If called by multiple threads, there will be a race on the creation of the tar file.

```
bool remote_install(const string& version);
```

Effects: If `version` refers to the file successfully downloaded by `remote_download()` this function will remove the existing time zone database, then extract a new database from the tar file, and will then delete the tar file.

This function *does not* cause your program to re-initialize itself from this new database. In order to do that, you must call `reload_tzdb()` (or `get_tzdb()` if the database has yet to be initialized).

Returns: `true` if the database was successfully replaced by the tar file , else `false`.

Thread safety: If called by multiple threads, there will be a race on the creation of the new database.

20.17.10.2 Exception classes [time.timezone.exception]

`nonexistent_local_time` is thrown when one attempts to convert a non-existent `local_time` to a `sys_time` without specifying `choose::earliest` or `choose::latest`.

```
class nonexistent_local_time
    : public runtime_error
{
public:
    // Construction is undocumented
};
```

[*Example:*

```
    #include "tz.h"
    #include <iostream>

    int
    main()
    {
        using namespace std::chrono;
        try
        {
            auto zt = make_zoned("America/New_York", local_days{sun[2]/mar/2016} + 2h + 30min);
        }
        catch (const nonexistent_local_time& e)
        {
            std::cout << e.what() << '\n';
        }
    }
```

Which outputs:

```
2016-03-13 02:30:00 is in a gap between
2016-03-13 02:00:00 EST and
2016-03-13 03:00:00 EDT which are both equivalent to
2016-03-13 07:00:00 UTC
```

— *end example:*]

`ambiguous_local_time` is thrown when one attempts to convert an ambiguous `local_time` to a `sys_time` without specifying `choose::earliest` or `choose::latest`.

```
class ambiguous_local_time
    : public runtime_error
{
public:
    // Construction is undocumented
};
```

[*Example:*

```
#include "tz.h"
#include <iostream>

int
main()
{
    using namespace std::chrono;
    try
    {
        auto zt = make_zoned("America/New_York", local_days{sun[1]/nov/2016} + 1h + 30min);
    }
    catch (const ambiguous_local_time& e)
    {
        std::cout << e.what() << '\n';
    }
}
```

Which outputs:

```
2016-11-06 01:30:00 is ambiguous.  It could be
2016-11-06 01:30:00 EDT == 2016-11-06 05:30:00 UTC or
2016-11-06 01:30:00 EST == 2016-11-06 06:30:00 UTC
```

— *end example:*]

20.17.10.3 Information classes [time.timezone.info]

A `sys_info` structure can be obtained from the combination of a `time_zone` and either a `sys_time`, or `local_time`. It can also be obtained from a `zoned_time` which is effectively a `pair` of a `time_zone` and `sys_time`.

This structure represents a lower-level API. Typical conversions from `sys_time` to `local_time` will use this structure *implicitly*, not *explicitly*.

```
struct sys_info
{
    sys_seconds   begin;
    sys_seconds   end;
    seconds       offset;
    minutes       save;
    string        abbrev;
};
```

The `begin` and `end` fields indicate that for the associated `time_zone` and `time_point`, the `offset` and `abbrev` are in effect in the range `[begin, end)`. This information can be used to efficiently iterate the transitions of a `time_zone`.

The `offset` field indicates the UTC offset in effect for the associated `time_zone` and `time_point`. The relationship between `local_time` and `sys_time` is:

```
offset = local_time - sys_time
```

The `save` field is "extra" information not normally needed for conversion between `local_time` and `sys_time`. If `save != 0min`, this `sys_info` is said to be on "daylight saving" time, and `offset - save` suggests what this `time_zone` *might* use if it were off daylight saving. However this information should not be taken as authoritative. The only sure way to get such information is to query the

time_zone with a time_point that returns an sys_info where save == 0min. There is no guarantee what time_point might return such an sys_info except that it is guaranteed *not* to be in the range [begin, end) (if save != 0min for this sys_info).

The abbrev field indicates the current abbreviation used for the associated time_zone and time_point. Abbreviations are not unique among the time_zones, and so one can not reliably map abbreviations back to a time_zone and UTC offset.

A sys_info can be streamed out in an unspecified format:

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const sys_info& r);
```

A local_info structure represents a lower-level API. Typical conversions from local_time to sys_time will use this structure *implicitly*, not *explicitly*.

```
struct local_info
{
    enum {unique, nonexistent, ambiguous} result;
    sys_info first;
    sys_info second;
};
```

When a local_time to sys_time conversion is unique, result == unique, first will be filled out with the correct sys_info and second will be zero-initialized. If the conversion stems from a nonexistent local_time then result == nonexistent, first will be filled out with the sys_info that ends just prior to the local_time and second will be filled out with the sys_info that begins just after the local_time. If the conversion stems from an ambiguous local_time then result == ambiguous, first will be filled out with the sys_info that ends just after the local_time and second will be filled out with the sys_info that starts just before the local_time.

A local_info can be streamed out in an unspecified format:

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const local_info& r);
```

20.17.10.4 Class `time_zone` [time.timezone.time_zone]

A time_zone represents all time zone transitions for a specific geographic area. time_zone construction is undocumented, and done during the database initialization. You can gain const access to a time_zone via functions such as locate_zone.

```
class time_zone
{
public:
    time_zone(const time_zone&) = delete;
    time_zone& operator=(const time_zone&) = delete;

    const string& name() const noexcept;

    template <class Duration> sys_info   get_info(sys_time<Duration> st)   const;
    template <class Duration> local_info get_info(local_time<Duration> tp) const;

    template <class Duration>
        sys_time<typename common_type<Duration, seconds>::type>
        to_sys(local_time<Duration> tp) const;

    template <class Duration>
        sys_time<typename common_type<Duration, seconds>::type>
        to_sys(local_time<Duration> tp, choose z) const;

    template <class Duration>
        local_time<typename common_type<Duration, seconds>::type>
        to_local(sys_time<Duration> tp) const;
};

bool operator==(const time_zone& x, const time_zone& y) noexcept;
bool operator!=(const time_zone& x, const time_zone& y) noexcept;
bool operator< (const time_zone& x, const time_zone& y) noexcept;
bool operator> (const time_zone& x, const time_zone& y) noexcept;
bool operator<=(const time_zone& x, const time_zone& y) noexcept;
bool operator>=(const time_zone& x, const time_zone& y) noexcept;

const string& time_zone::name() const noexcept;
```

Returns: The name of the time_zone.

Example: "America/New_York".

Note: Here is an unofficial list of `time_zone` names: https://en.wikipedia.org/wiki/List_of_tz_database_time_zones.

```
template <class Duration> sys_info time_zone::get_info(sys_time<Duration> st) const;
```

Returns: A `sys_info i` for which `st` is in the range `[i.begin, i.end)`.

```
template <class Duration> local_info time_zone::get_info(local_time<Duration> tp) const;
```

Returns: A `local_info` for `tp`.

```
template <class Duration>
sys_time<typename common_type<Duration, seconds>::type>
time_zone::to_sys(local_time<Duration> tp) const;
```

Returns: A `sys_time` that is at least as fine as `seconds`, and will be finer if the argument `tp` has finer precision. This `sys_time` is the UTC equivalent of `tp` according to the rules of this `time_zone`.

Throws: If the conversion from `tp` to a `sys_time` is ambiguous, throws `ambiguous_local_time`. If the conversion from `tp` to a `sys_time` is nonexistent, throws `nonexistent_local_time`.

```
template <class Duration>
sys_time<typename common_type<Duration, seconds>::type>
time_zone::to_sys(local_time<Duration> tp, choose z) const;
```

Returns: A `sys_time` that is at least as fine as `seconds`, and will be finer if the argument `tp` has finer precision. This `sys_time` is the UTC equivalent of `tp` according to the rules of this `time_zone`. If the conversion from `tp` to a `sys_time` is ambiguous, returns the earlier `sys_time` if `z == choose::earliest`, and returns the later `sys_time` if `z == choose::latest`. If the `tp` represents a non-existent time between two UTC `time_point`s, then the two UTC `time_point`s will be the same, and that UTC `time_point` will be returned.

```
template <class Duration>
local_time<typename common_type<Duration, seconds>::type>
time_zone::to_local(sys_time<Duration> tp) const;
```

Returns: The `local_time` associated with `tp` and this `time_zone`.

```
bool operator==(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `x.name() == y.name()`.

```
bool operator!=(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `!(x == y)`.

```
bool operator<(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `x.name() < y.name()`.

```
bool operator>(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `y < x`.

```
bool operator<=(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `!(y < x)`.

```
bool operator>=(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `!(x < y)`.

20.17.10.5 Class `zoned_time` [time.timezone.zoned_time]

`zoned_time` represents a logical paring of `time_zone` and a `time_point` with precision `Duration`. If `seconds` is not implicitly convertible to `Duration`, the instantiation is ill-formed. [*Note:* There exist `time_zone`s with UTC offsets that require a precision of `seconds`. — *end note:*]

```
template <class Duration>
class zoned_time
{
    const time_zone*   zone_;  // exposition only
    sys_time<Duration> tp_;    // exposition only

public:
    zoned_time(const zoned_time&) = default;
    zoned_time& operator=(const zoned_time&) = default;
```

```
            zoned_time(const sys_time<Duration>& st);
    explicit zoned_time(const time_zone* z);
    explicit zoned_time(const string& name);

    template <class Duration2>
        zoned_time(const zoned_time<Duration2>& zt) noexcept;

    zoned_time(const time_zone* z, const local_time<Duration>& tp);
    zoned_time(const string& name, const local_time<Duration>& tp);
    zoned_time(const time_zone* z, const local_time<Duration>& tp, choose c);
    zoned_time(const string& name, const local_time<Duration>& tp, choose c);

    zoned_time(const time_zone* z, const zoned_time<Duration>& zt);
    zoned_time(const string& name, const zoned_time<Duration>& zt);
    zoned_time(const time_zone* z, const zoned_time<Duration>& zt, choose);
    zoned_time(const string& name, const zoned_time<Duration>& zt, choose);

    zoned_time(const time_zone* z, const sys_time<Duration>& st);
    zoned_time(const string& name, const sys_time<Duration>& st);

    zoned_time& operator=(const sys_time<Duration>& st);
    zoned_time& operator=(const local_time<Duration>& ut);

            operator sys_time<Duration>()   const;
    explicit operator local_time<Duration>() const;

    const time_zone*     get_time_zone()  const;
    local_time<Duration> get_local_time() const;
    sys_time<Duration>   get_sys_time()   const;
    sys_info             get_info()       const;
};

template <class Duration1, class Duration2>
bool
operator==(const zoned_time<Duration1>& x, const zoned_time<Duration2>& y);

template <class Duration1, class Duration2>
bool
operator!=(const zoned_time<Duration1>& x, const zoned_time<Duration2>& y);
```

An invariant of `zoned_time<Duration>` is that it always refers to a valid `time_zone`, and represents a point in time that exists and is not ambiguous.

```
zoned_time<Duration>::zoned_time(const zoned_time&) = default;
zoned_time<Duration>& zoned_time<Duration>::operator=(const zoned_time&) = default;
```

> The copy members transfer the associated `time_zone` from the source to the destination. After copying, source and destination compare equal. If `Duration` has `noexcept` copy members, then `zoned_time<Duration>` has `noexcept` copy members.

```
zoned_time<Duration>::zoned_time(const sys_time<Duration>& st);
```

> *Effects:* Constructs a `zoned_time` `zt` such that `zt.get_time_zone()->name() == "UTC"`, and `zt.get_sys_time() == st`.

```
explicit zoned_time<Duration>::zoned_time(const time_zone* z);
```

> *Requires:* `z` refers to a valid `time_zone`.
>
> *Effects:* Constructs a `zoned_time` `zt` such that `zt.get_time_zone()-> == z`, and `zt.get_sys_time() == sys_seconds{}`.

```
explicit zoned_time<Duration>::zoned_time(const string& name);
```

> *Effects:* Equivalent to construction with `locate_zone(name)`.
>
> *Throws:* Any exception propagating out of `locate_zone(name)`.

```
template <class Duration2>
    zoned_time<Duration>::zoned_time(const zoned_time<Duration2>& y) noexcept;
```

> *Remarks:* Does not participate in overload resolution unless `sys_time<Duration2>` is implicitly convertible to `sys_time<Duration>`.
>
> *Effects:* Constructs a `zoned_time` `x` such that `x == y`.

```
zoned_time<Duration>::zoned_time(const time_zone* z, const local_time<Duration>& tp);
```

Requires: `z` refers to a valid `time_zone`.

Effects: Constructs a `zoned_time` `zt` such that `zt.get_time_zone()-> == z`, and `zt.get_local_time() == tp`.

Throws: Any exception that `z->to_sys(tp)` would throw.

```
zoned_time<Duration>::zoned_time(const string& name, const local_time<Duration>& tp);
```

Effects: Equivalent to construction with `{locate_zone(name), tp}`.

```
zoned_time<Duration>::zoned_time(const time_zone* z, const local_time<Duration>& tp, choose c);
```

Requires: `z` refers to a valid `time_zone`.

Effects: Constructs a `zoned_time` `zt` such that `zt.get_time_zone()-> == z`, and `zt.get_sys_time() == z->to_sys(tp, c)`.

```
zoned_time<Duration>::zoned_time(const string& name, const local_time<Duration>& tp, choose c);
```

Effects: Equivalent to construction with `{locate_zone(name), tp, c}`.

```
zoned_time<Duration>::zoned_time(const time_zone* z, const zoned_time<Duration>& y);
```

Requires: `z` refers to a valid `time_zone`.

Effects: Constructs a `zoned_time` `zt` such that `zt.get_time_zone()-> == z`, and `zt.get_sys_time() == y.get_sys_time()`.

```
zoned_time<Duration>::zoned_time(const string& name, const zoned_time<Duration>& y);
```

Effects: Equivalent to construction with `{locate_zone(name), y}`.

```
zoned_time<Duration>::zoned_time(const time_zone* z, const zoned_time<Duration>& y, choose);
```

Requires: `z` refers to a valid `time_zone`.

Effects: Constructs a `zoned_time` `zt` such that `zt.get_time_zone()-> == z`, and `zt.get_sys_time() == y.get_sys_time()`.

Note: The `choose` parameter is allowed here, but has no impact.

```
zoned_time<Duration>::zoned_time(const string& name, const zoned_time<Duration>& y, choose);
```

Effects: Equivalent to construction with `{locate_zone(name), y}`.

Note: The `choose` parameter is allowed here, but has no impact.

```
zoned_time<Duration>::zoned_time(const time_zone* z, const sys_time<Duration>& st);
```

Requires: `z` refers to a valid `time_zone`.

Effects: Constructs a `zoned_time` `zt` such that `zt.get_time_zone()-> == z`, and `zt.get_sys_time() == st`.

```
zoned_time<Duration>::zoned_time(const string& name, const sys_time<Duration>& st);
```

Effects: Equivalent to construction with `{locate_zone(name), st}`.

```
zoned_time<Duration>& zoned_time<Duration>::operator=(const sys_time<Duration>& st);
```

Effects: After assignment `get_sys_time() == st`. This assignment has no effect on the return value of `get_time_zone()`.

Returns: `*this`.

```
zoned_time<Duration>& zoned_time<Duration>::operator=(const local_time<Duration>& lt);
```

Effects: After assignment `get_local_time() == lt`. This assignment has no effect on the return value of `get_time_zone()`.

Returns: `*this`.

```
zoned_time<Duration>::operator sys_time<Duration>() const;
```

Returns: `get_sys_time()`.

```
explicit zoned_time<Duration>::operator local_time<Duration>() const;
```

Returns: `get_local_time()`.

```
const time_zone* zoned_time<Duration>::get_time_zone() const;
```

Returns: `zone_`.

```
local_time<Duration> zoned_time<Duration>::get_local_time() const;
```

Returns: `zone_->to_local(tp_)`.

```
sys_time<Duration> zoned_time<Duration>::get_sys_time() const;
```

Returns: `tp_`.

```
sys_info zoned_time<Duration>::get_info() const;
```

Returns: `zone_->get_info(tp_)`.

```
template <class Duration1, class Duration2>
bool
operator==(const zoned_time<Duration1>& x, const zoned_time<Duration2>& y);
```

Returns: `x.zone_ == y.zone_ && x.tp_ == y.tp_`.

```
template <class Duration1, class Duration2>
bool
operator!=(const zoned_time<Duration1>& x, const zoned_time<Duration2>& y);
```

Returns: `!(x == y)`.

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const zoned_time<Duration>& t)
```

Effects: Streams `t` to `os` using the format "%F %T %Z" and the value returned from `t.get_local_time()`.

Returns: `os`.

20.17.10.6 Helper functions `make_zoned` [time.timezone.make_zoned]

There exist several overloaded functions named `make_zoned` which serve as factory functions for `zoned_time<Duration>` and will deduce the correct `Duration` from the argument list. In every case the correct return type is `zoned_time<common_type_t<Duration, seconds>>`.

```
template <class Duration>
zoned_time<common_type_t<Duration, seconds>>
make_zoned(const sys_time<Duration>& tp)
```

Returns: `{tp}`.

```
template <class Duration>
zoned_time<common_type_t<Duration, seconds>>
make_zoned(const time_zone* zone, const local_time<Duration>& tp)
```

Returns: `{zone, tp}`.

```
template <class Duration>
zoned_time<common_type_t<Duration, seconds>>
make_zoned(const string& name, const local_time<Duration>& tp)
```

Returns: `{name, tp}`.

```
template <class Duration>
zoned_time<common_type_t<Duration, seconds>>
make_zoned(const time_zone* zone, const local_time<Duration>& tp, choose c)
```

Returns: `{zone, tp, c}`.

```
template <class Duration>
zoned_time<common_type_t<Duration, seconds>>
make_zoned(const string& name, const local_time<Duration>& tp, choose c)
```

Returns: `{name, tp, c}`.

```
template <class Duration>
zoned_time<common_type_t<Duration, seconds>>
make_zoned(const time_zone* zone, const zoned_time<Duration>& zt)
```

Returns: `{zone, zt}`.

```
template <class Duration>
zoned_time<common_type_t<Duration, seconds>>
make_zoned(const string& name, const zoned_time<Duration>& zt)
```

Returns: `{name, zt}`.

```
template <class Duration>
zoned_time<common_type_t<Duration, seconds>>
make_zoned(const time_zone* zone, const zoned_time<Duration>& zt, choose c)
```

Returns: `{zone, zt, c}`.

```
template <class Duration>
zoned_time<common_type_t<Duration, seconds>>
make_zoned(const string& name, const zoned_time<Duration>& zt, choose c)
```

Returns: `{name, zt, c}`.

```
template <class Duration>
zoned_time<common_type_t<Duration, seconds>>
make_zoned(const time_zone* zone, const sys_time<Duration>& st)
```

Returns: `{zone, st}`.

```
template <class Duration>
zoned_time<common_type_t<Duration, seconds>>
make_zoned(const string& name, const sys_time<Duration>& st)
```

Returns: `{name, st}`.

20.17.10.7 Formatting [time.timezone.format]

```
template <class CharT, class Traits, class Rep, class Period>
void
to_stream(basic_ostream<CharT, Traits>& os, const CharT* fmt, const duration<Rep, Period>& d);
```

Effects: Streams `d` into `os` using the format specified by the null-terminated array `fmt`. `fmt` encoding follows the rules of `strftime` except that only the following are recognized as commands:

```
%H, %I, %M, %n, %p, %r, %R, %S, %t, %T, %X, %%
```

Some of these can be modified by `E` or `O` as specified by `strftime`.

For the commands `%S` and `%T`, if `d` has precision that is not an integral number of seconds, then seconds are formatted as a decimal floating point number with a fixed format and a precision showing the full precision of `d` if that can be represented by 18 or fewer fractional decimal digits. If the full precision of `d` can not be represented, then 6 fractional decimal digits will be output. In either case "precision" refers to all representable values, including trailing zeroes.

[*Example:*

```
using quarter_sec = duration<int, ratio<1, 4>>;
to_stream(cout, "%T", quarter_sec{15002});  // 01:02:30.50
// Two digits of precision can represent all quarter_sec values
```

— end example]

```
template <class CharT, class Traits, class Duration>
void
to_stream(basic_ostream<CharT, Traits>& os, const CharT* fmt,
          const local_time<Duration>& tp, const string* abbrev = nullptr,
          const seconds* offset_sec = nullptr);
```

Effects: Streams `tp` into `os` using the format specified by the null-terminated array `fmt`. `fmt` encoding follows the rules of `strftime`. Those formatting commands pertaining to time of day are formatted according to the rules specified by the `to_stream` overload taking a duration formed by extracting the time of day from `tp`.

If `%Z` is used and `abbrev` is not `nullptr`, then `*abbrev` is streamed into `os`. Otherwise if `%Z` is used and `abbrev == nullptr` then a `runtime_error` is thrown with an unspecified `what()` message.

If `%z` is used and `offset_sec` is not `nullptr`, then `*offset_sec` is streamed into `os` with the format `hhmm`. If the command is modified by either `E` or `O`, then a colon (`':'`) is inserted between the hours and minutes. Otherwise if `%z` is used and `offset_sec == nullptr` then a `runtime_error` is thrown with an unspecified `what()` message.

```
template <class CharT, class Traits, class Duration>
```

```
void
to_stream(basic_ostream<CharT, Traits>& os, const CharT* fmt, const sys_time<Duration>& tp);
```

Effects: Equivalent to:

```
          const string abbrev("UTC");
          constexpr seconds offset{0};
          to_stream(os, fmt, local_time<Duration>{tp.time_since_epoch()}, &abbrev, &offset);
```

```
template <class CharT, class Traits, class Duration>
void
to_stream(basic_ostream<CharT, Traits>& os, const CharT* fmt, const zoned_time<Duration>& tp);
```

Effects: Equivalent to:

```
          auto const info = tp.get_info();
          to_stream(os, fmt, tp.get_local_time(), &info.abbrev, &info.offset);
```

```
// format duration

template <class charT, class traits, class Rep, class Period>
basic_string<charT, traits>
format(const locale& loc, const basic_string<charT, traits>& fmt, const duration<Rep, Period>& d);

template <class charT, class traits, class Rep, class Period>
basic_string<charT, traits>
format(const basic_string<charT, traits>& fmt, const duration<Rep, Period>& d);

template <class charT, class Rep, class Period>
basic_string<charT>
format(const locale& loc, const charT* fmt, const duration<Rep, Period>& d);

template <class charT, class Rep, class Period>
basic_string<charT>
format(const charT* fmt, const duration<Rep, Period>& d);

// format local_time

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const locale& loc, const basic_string<class charT, class traits>& format,
       const local_time<Duration>& tp);

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const basic_string<class charT, class traits>& format, const local_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const locale& loc, const charT* format, const local_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const charT* format, const local_time<Duration>& tp);

// format sys_time

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const locale& loc, const basic_string<class charT, class traits>& format,
       const sys_time<Duration>& tp);

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const basic_string<class charT, class traits>& format, const sys_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const locale& loc, const charT* format, const sys_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const charT* format, const sys_time<Duration>& tp);

// format zoned_time

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const locale& loc, const basic_string<class charT, class traits>& format,
```

```
      const zoned_time<Duration>& tp);

template <class charT, class traits, class Duration>
basic_string<class charT, class traits>
format(const basic_string<class charT, class traits>& format, const zoned_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const locale& loc, const charT* format, const zoned_time<Duration>& tp);

template <class charT, class Duration>
basic_string<class charT>
format(const charT* format, const zoned_time<Duration>& tp);
```

Effects: Creates a local `basic_ostringstream<charT, traits>` (defaulting `traits` when `format` is a `const charT*`). If a `locale` is passed in, imbues the `basic_ostringstream` with this `locale`. Then calls `to_stream` using the `basic_ostringstream`, `format.c_str()` (or `format` as appropriate), and the `duration`, `time_point` or `zoned_time`.

Returns: The formatted string from the local `basic_ostringstream`.

20.17.10.8 Parsing [time.timezone.parse]

```
template <class Rep, class Period, class CharT, class Traits>
unspecified
parse(const basic_string<CharT, Traits>& format, duration<Rep, Period>& d);

template <class Rep, class Period, class CharT>
unspecified
parse(const CharT* format, duration<Rep, Period>& d);
```

Returns: An object of unspecified type with a stream extraction operator that behaves as a formatted input function and attempts to parse a `duration` out of the input stream `is` according to `format`.

The `format` string follows the rules as specified for `time_get` with the following exceptions:

- Only the following are recognized as commands: `%H`, `%I`, `%M`, `%n`, `%p`, `%r`, `%R`, `%S`, `%t`, `%T`, `%X`, `%%`.
- If `%S` or `%T` appears in the `format` string and the argument `d` has precision that is not an integral number of seconds, then the seconds are parsed as a `double`, and if that parse is successful contributes to the time stamp as if `round<Duration>(duration<double>{s})` where `s` is a local variable holding the parsed `double`.

One can parse in a `sys_time<Duration>` or a `local_time<Duration>`. Optionally, one can also pass in a reference to a `string` in order to capture the time zone abbreviation, or one can pass in a reference to a `minutes` to capture a time zone UTC offset (formatted as specified by `%z` or its modified variants), or one can pass in both in either order.

```
// parse sys_time

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, sys_time<Duration>& tp);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, sys_time<Duration>& tp,
      basic_string<charT, traits>& abbrev);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, sys_time<Duration>& tp,
      minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, sys_time<Duration>& tp,
      basic_string<charT, traits>& abbrev, minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, sys_time<Duration>& tp,
      minutes& offset, basic_string<charT, traits>& abbrev);

template <class Duration, class charT>
unspecified
```

```
parse(const charT* format, sys_time<Duration>& tp);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, sys_time<Duration>& tp,
      basic_string<charT, traits>& abbrev);

template <class Duration, class charT>
unspecified
parse(const charT* format, sys_time<Duration>& tp,
      minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, sys_time<Duration>& tp,
      basic_string<charT, traits>& abbrev, minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, sys_time<Duration>& tp,
      minutes& offset, basic_string<charT, traits>& abbrev);

// parse local_time

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, local_time<Duration>& tp);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, local_time<Duration>& tp,
      basic_string<charT, traits>& abbrev);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, local_time<Duration>& tp,
      minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, local_time<Duration>& tp,
      basic_string<charT, traits>& abbrev, minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const basic_string<charT, traits>& format, local_time<Duration>& tp,
      minutes& offset, basic_string<charT, traits>& abbrev);

template <class Duration, class charT>
unspecified
parse(const charT* format, local_time<Duration>& tp);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, local_time<Duration>& tp,
      basic_string<charT, traits>& abbrev);

template <class Duration, class charT>
unspecified
parse(const charT* format, local_time<Duration>& tp,
      minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, local_time<Duration>& tp,
      basic_string<charT, traits>& abbrev, minutes& offset);

template <class Duration, class charT, class traits>
unspecified
parse(const charT* format, local_time<Duration>& tp,
      minutes& offset, basic_string<charT, traits>& abbrev);
```

Returns: These functions return an object of unspecified type with a stream extraction operation that behaves as a formatted input function and attempts to parse a `time_point` out of the input stream `is` according to `format`.

The `format` string follows the rules as specified for `time_get` with the following exceptions:

- If `%F` appears in the format string it is interpreted as `%Y-%m-%d`.

- If %S or %T appears in the format string and the argument tp has precision that is not an integral number of seconds, then the seconds are parsed as a double, and if that parse is successful contributes to the time stamp as if round<Duration>(duration<double>{s}) where s is a local variable holding the parsed double.

- If %z appears in the format string and an offset is successfully parsed, the overloads taking sys_time interprets the parsed time as a local time and subtracts the offset prior to assigning the value to tp, resulting in a value of tp representing a UTC timestamp. The overloads taking local_time require a valid parse of the offset, but then ignore the offset in assigning a value to the local_time<Duration>& tp. If offset is passed in, on successful parse it will hold the value represented by %z if present, or will be assigned 0min if %z is not present.

  The format of the offset is +/-hhmm. The leading plus or minus sign is required. If the format string was modified (i.e. %Ez or %Oz), a colon is required between hours and minutes, and the leading hours digit is optional: +/-[h]h:mm.

- If %Z appears in the format string then an abbreviation is required in that position for a successful parse. The abbreviation will be parsed as a string (delimited by white space). The parsed abbreviation does not have to be a valid time zone abbreviation, and has no impact on the value parsed into tp. Using the overloads that take a string& one can discover what that parsed abbreviation is. On successful parse, abbrev will be assigned the value represented by %Z if present, or assigned the empty string if %Z is not present.

[*Example:*

```
sys_seconds tp;
minutes offset;
istringstream in("2016-10-02 22:54:00 -0400");
in >> parse("%F %T %z", tp, offset);
// tp == 1475463240s
// offset == -240min
```

— *end example*]

20.17.10.10 class `leap` [time.timezone.leap]

```
class leap
{
public:
    leap(const leap&)            = default;
    leap& operator=(const leap&) = default;

    // Undocumented constructors

    sys_seconds date() const;
};

bool operator==(const leap& x, const leap& y);
bool operator!=(const leap& x, const leap& y);
bool operator< (const leap& x, const leap& y);
bool operator> (const leap& x, const leap& y);
bool operator<=(const leap& x, const leap& y);
bool operator>=(const leap& x, const leap& y);

template <class Duration> bool operator==(const const leap&         x, const sys_time<Duration>& y);
template <class Duration> bool operator==(const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator!=(const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator!=(const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator< (const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator< (const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator> (const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator> (const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator<=(const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator<=(const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator>=(const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator>=(const sys_time<Duration>& x, const leap&               y);
```

leap is a copyable class that is constructed and stored in the time zone database when initialized. You can explicitly convert it to a sys_seconds with the member function date() and that will be the date of the leap second insertion. leap is equality and less-than comparable, both with itself, and with sys_time<Duration>.

20.17.10.11 class `link` [time.timezone.link]

```
class link
{
public:
    link(const link&)            = default;
    link& operator=(const link&) = default;

    // Undocumented constructors

    const string& name()   const;
    const string& target() const;
};

bool operator==(const link& x, const link& y);
bool operator!=(const link& x, const link& y);
bool operator< (const link& x, const link& y);
bool operator> (const link& x, const link& y);
bool operator<=(const link& x, const link& y);
bool operator>=(const link& x, const link& y);
```

A `link` is an alternative name for a `time_zone`. The alternative name is `name()`. The name of the `time_zone` for which this is an alternative name is `target()`. `link`s will be constructed for you when the time zone database is initialized.

Acknowledgements

A database parser is nothing without its database. I would like to thank the founding contributor of the IANA Time Zone Database Arthur David Olson. I would also like to thank the entire group of people who continually maintain it, and especially the IESG-designated TZ Coordinator, Paul Eggert. Without the work of these people, this software would have no data to parse.

I would also like to thank Jiangang Zhuang and Bjarne Stroustrup for invaluable feedback for the timezone portion of this library, which ended up also influencing the date.h library. Thanks also to Jonathan Wakely for agreeing to present this paper in Oulu for me. Thank you Daniel Krügler for the incredibly thorough review.

And I would also especially like to thank contributors to this library: gmcode, Ivan Pizhenko, tomy2105 and Ville Voutilainen.

References

1. N. Dershowitz and E. Reingold, *Calendrical Calculations 3rd ed.*, Cambridge University Press 2008.